université
PARIS-SACLAY

RAPPORT DE TP

Commande numérique pour l'asservissement en position d'un servomécanisme

Auteur :

Gaëtan BOUCHY
Elie NSEIR

Enseignant :

M. TLIBA

Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun en binôme et qu'il a été rédigé de manière autonome.

À Orsay, le 08/03/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
Partie Théorique 4
1. Construction d'un modèle de connaissance du système 4
1.1 Analyse du fonctionnement 4
1.2 Schéma-bloc 6
1.2.1 Domaine temporel 6
1.2.2 Domaine de Laplace 6
1.3 Détermination des fonctions de transfert 7
1.3.1 Donner l'expression de $\mathbf{Pr(t)}$. En déduire sa transformée de Laplace 7
1.3.2 Hypothèse sur le fonctionnement du système (conditions initiales, paramètres négligeables) et calcul des transformées de Laplace 8
1.3.3 Calculer toutes les fonctions de transfert recherchées 9
1.3.4 On suppose que le frottement visqueux est négligeable $(\boldsymbol{f} \approx \mathbf{0})$ 10
1.3.5 Faire de même en supposant $\boldsymbol{Rc} = \mathbf{0}$ et $\boldsymbol{Rc} \to +\infty$ 11
1.4 Propriétés dynamiques 12
2. Simulations numériques 13
2.1 Saisie des différents paramètres et des données numériques 13
2.2 Construire le schéma-bloc Simulink représentant le modèle complet du servomécanisme 14
2.3 Pour $\boldsymbol{f} = \mathbf{0}$ et $\boldsymbol{Rc} = \mathbf{0}$, étudier l'influence de $\mathbf{Cr0}$ sur le temps de réponse et le gain statique 15
2.4 Étudier l'influence de $\boldsymbol{f}$ et $\boldsymbol{Rc}$ de sur le temps de réponse et le gain statique 18
2.5 Calcul des pôles et évolution dans le plan complexe 29
2.6 Diagramme de Bode du transfert de U(p) vers $U_{GT}(p)$ 32
2.7 Diagramme de Bode du transfert de $P_r(p)$ vers $U_{GT}(p)$ 33
2.8 Interprétation et conclusion 34
3. Travail demandé 35
3.1 Transfert de vitesse 35
3.1.1 Représentez le schéma-bloc du correcteur RST 35
3.1.2 Établir un cahier des charges 36
3.1.3 Calcul de l'ordre minimal du correcteur RST ainsi que les valeurs numériques des polynômes R(z), S(z) et T(z) 36
Partie Pratique 37

4 Mise en œuvre pratique 37
4.1 L'identification du système 37
4.2 Modèle de commande du système 42

université PARIS-SACLAY

Partie Théorique

1. Construction d'un modèle de connaissance du système

1.1 Analyse du fonctionnement

Voici le tableau de toute les grandeurs physiques qui peuvent être considérées comme des commandes, des sorties mesurées, des perturbations du système ou des sorties que l'on souhaite contrôler par l'asservissement :

Caractéristique / Grandeur physique	Des commandes	Des sorties mesurées	Des perturbations du système	Des sorties que l'on souhaite contrôler par l'asservissement
U(t), tension circulant dans le bobinage	X			
$I_m(t)$, courant circulant dans le bobinage	X			
$E_m(t)$, force contre électromotrice du moteur				X
$\Omega_m(t)$, vitesse angulaire du moteur		X		
$C_m(t)$, couple mécanique produit par le moteur				X
$U_{GC}(t)$, tension circulant dans la génératrice de courant continu	X			
$I_g(t)$, courant circulant dans la génératrice de courant continu	X			
$E_g(t)$, force contre électromotrice de la génératrice de courant continu				X
$C_g(t)$, couple mécanique produit par la génératrice de courant continu				X
$C_{f1}(t)$, couple de frottement sec			X	

$C_{f2}(t)$, couple de frottement visqueux			X	
$C_{r1}(t)$, couple de frottement généré par les roulements mécaniques			X	
$f^*\Omega_r(t)$, terme de frottement visqueux			X	
$\Omega_r(t)$, vitesse angulaire du rotor		X		
$C_{r2}(t)$, terme de frottement sec			X	

Tableau 1 : Tableau des grandeurs physiques en fonction de leur caractéristique

La grandeur physique U(t), tension circulant dans le bobinage peut être considérée comme une commande qui est associé à l'amplificateur de puissance. Idem pour la génératrice de courant continu.

Le courant circulant dans le bobinage, $I_m(t)$ peut être considéré comme une commande qui est associée à l'amplificateur de puissance. Idem pour la génératrice de courant continu.

La grandeur physique $\Omega_m(t)$, la vitesse angulaire du moteur, peut être considérée comme une sortie mesurée par des capteurs. La vitesse angulaire peut être mesurée à l'aide de capteurs tels que des encodeurs optiques, des codeurs incrémentaux ou des tachymètres. Idem pour la génératrice de courant continu et la vitesse angulaire du rotor.

1.2 Schéma-bloc

1.2.1 Domaine temporel

À partir des équations suivantes, nous pouvons aboutir au schéma-bloc :

- $\frac{dIm(t)}{dt} = -\frac{Ke}{L} * \Omega_m(t) - \frac{R}{L} * I_m(t) + \frac{1}{L} * U(t)$
- $\frac{d\Omega_m(t)}{dt} = \frac{Kc}{J} * I_m(t) - \frac{1}{J} * (Cf_1(t) + Cf_2(t) + C_g(t)$
- $\frac{dIm(t)}{dt} = \frac{1}{L} * E_g - \frac{\tilde{R}}{L} * I_g - \frac{1}{L} * U_{GC}$

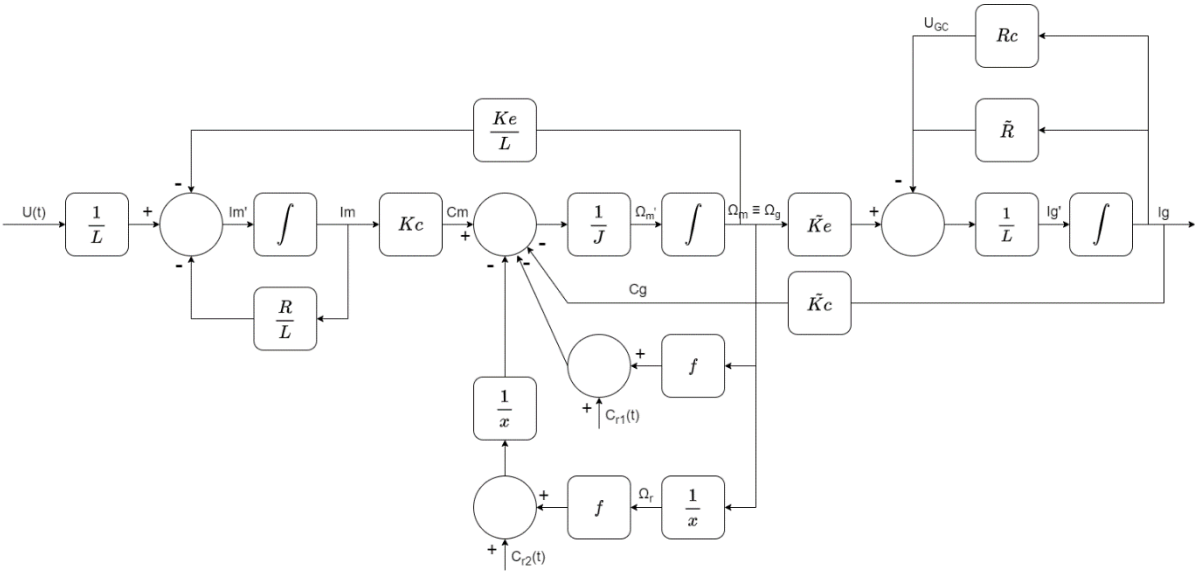


Figure 1 : Schéma-Bloc dans le domaine temporel

1.2.2 Domaine de Laplace

Nous pouvons simplement convertir le schéma-bloc dans le domaine temporel en domaine de Laplace en remplaçant les opérations dans le temps par leurs transformations de Laplace. Notamment, l'intégration dans le domaine temporel correspond à une division par p dans le domaine de Laplace :

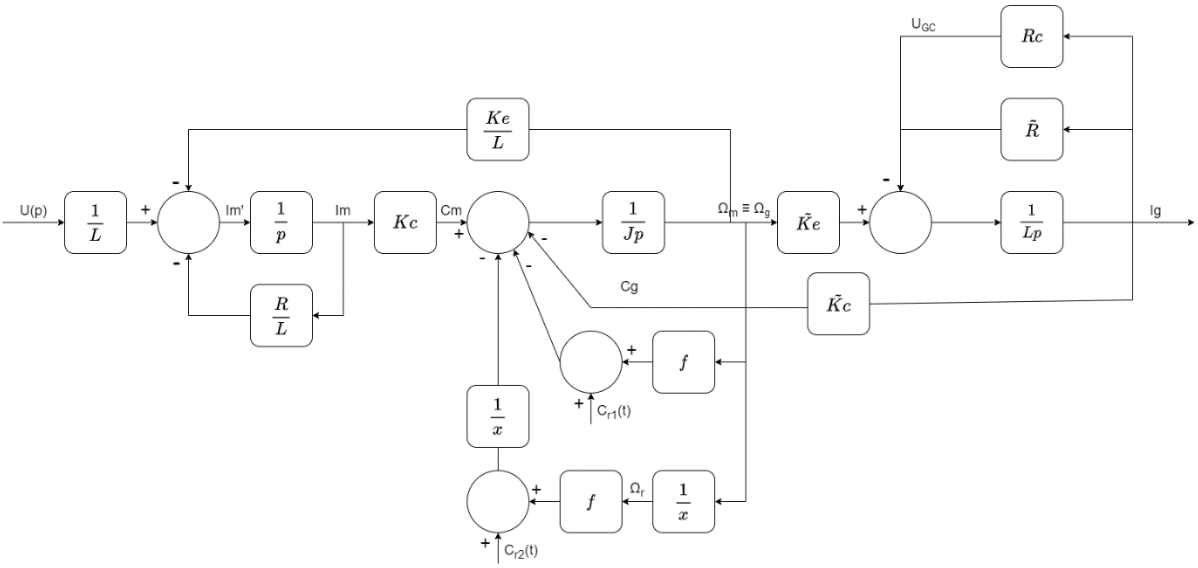


Figure 2 : Schéma-Bloc dans le domaine de Laplace

1.3 Détermination des fonctions de transfert

1.3.1 Donner l'expression de $\mathrm{P_r(t)}$. En déduire sa transformée de Laplace

On sait d'après l'énoncé que $\mathrm{P_r(t)}$ correspond aux perturbations statiques (donc aux frottements secs) ramenés à l'entrée et que :

$C_{f1}(t) = f * \Omega_m(\mathrm{t}) + \mathrm{C}_{r1}(t)$, $C_{f2}(t) = f * \Omega_r(\mathrm{t}) + \mathrm{C}_{r2}(t)$, $\Omega_r = \frac{1}{x} * \Omega_m$ d'après le principe du réducteur et $J * \frac{d*\Omega_m(\mathrm{t})}{dt} = C_m(t) - C_g(t) - C_{f1}(t) - \frac{1}{x} * C_{f2}(t)$ avec $\mathrm{C}_{r1}(t) = \mathrm{C}_{r2}(t) = \mathrm{C}_{r0}$.

On a donc $P_r(t) = \int(\mathrm{C}_{r1}(t) + \frac{1}{x} * \mathrm{C}_{r2}(t)) = \int C_{r0} * (1 + \frac{1}{x})$

De ce fait, nous obtenons l'expression suivante pour $P_r(p)$: $P_r(p) = \frac{C_{r0}*(1+\frac{1}{x})}{p}$ car une intégrale dans le domaine temporel équivaut à une division par p dans le domaine de Laplace. Le servo-mécanisme est un polynôme d'ordre 3 ce qui implique que la réponse des frottements est régie par le premier terme.

1.3.2 Hypothèse sur le fonctionnement du système (conditions initiales, paramètres négligeables) et calcul des transformées de Laplace

D'après l'énoncé, nous pouvons remarquer différentes hypothèses :
- L'amplificateur de puissance est supposé parfaitement linéaire en dehors du régime de saturation en courant et en tension
- Le moteur et la génératrice sont des moteurs à courant continu identiques
- Les moments d'inerties :
  - Réducteur du moto-réducteur : J_R supposé très faible devant J_{MCC}
  - Génératrice tachymétrique : J_{GT} supposé très faible devant J_{MCC}
  - Potentiomètre : J_P supposé très faible devant J_{MCC}. Le gain du potentiomètre est parfaitement linéaire tant que le courant qu'il délivre est inférieur à 2,5A.
- $\mathrm{C}_{r1}(t) = \mathrm{C}_{r2}(t) = \mathrm{C}_{r0} \ \forall t \in \mathbb{R}^+ / \Omega_m(t) \neq 0$
- L'instant initial considéré est $t_0 = 0$. On suppose que le système est au repos avant cet instant et qu'il ne l'est plus à partir de cet instant (principe de causalité)

On peut ainsi réaliser le calcul des transformées de Laplace aux équations issues de l'énoncé (établies à la section 2) :

- $U(p) = \mathrm{E}_m(p) + \mathrm{I}_m(p) * (R + L * p)$, une intégrale dans le domaine temporel équivaut à une division par p dans le domaine de Laplace et une dérivation dans le domaine temporel équivaut à une multiplication par p dans le domaine de Laplace (1)
- $\mathrm{E}_m(p) = \mathrm{K}_e * \Omega_m(p)$ (2)
- $\mathrm{C}_m(p) = \mathrm{K}_c * \mathrm{I}_m(p)$ (3)
- $\mathrm{U}_{GC}(p) = \mathrm{E}_g(p) + \mathrm{I}_g(p) * (\tilde{R} + L * p)$ (4)
- $\mathrm{U}_{GC}(p) = R_c * \mathrm{I}_g(p)$ (5)
- $\mathrm{E}_g(p) = \widetilde{\mathrm{K}_e} * \Omega_m(p)$ (6)
- $\mathrm{C}_g(p) = \widetilde{\mathrm{K}_c} * \mathrm{I}_g(p)$ (7)
- $J * p * \Omega_m(p) = C_m(p) - C_g(p) - C_{f1}(p) - \frac{1}{x} * C_{f2}(p)$ (8)
- $C_{f1}(p) = f * \Omega_m(\mathrm{p}) + \mathrm{C}_{r1}(p)$ (10)
- $C_{f2}(p) = f * \Omega_r(\mathrm{p}) + \mathrm{C}_{r2}(p)$ (11)
- $\Omega_r(\mathrm{p}) = \frac{1}{x} * \Omega_m(p)$ (12)
- $\mathrm{C}_{r1}(p) = \mathrm{C}_{r2}(p) = \mathrm{C}_{r0}$ (13)

1.3.3 Calculer toutes les fonctions de transfert recherchées

D'après les équations mécaniques et électriques du moteur ainsi que l'équation régissant le fonctionnement de la génératrice de courant :

$$J * \frac{d\Omega_m(\mathrm{t})}{dt} = C_m(t) - C_g(t) - C_{f1}(t) - \frac{1}{x} * C_{f2}(t)$$

$$\frac{dI_m(t)}{dt} = -\frac{Ke}{L} * \Omega_m(t) - \frac{R}{L} * I_m(t) + \frac{1}{L} * U(t)$$

$$\frac{dI_g(t)}{dt} = -\frac{\tilde{K}_e}{\tilde{L}} * \Omega_m(t) - \frac{\tilde{R}}{\tilde{L}} * I_g(t) + \frac{1}{\tilde{L}} R_C * I_g(t)$$

Le moteur et la génératrice étant des machines identiques, leurs paramètres physiques sont identiques :

$R = \tilde{R}, \quad L = \tilde{L}, \quad \tilde{K}_C = K_C, \quad \tilde{K}_e = K_e$

On calcule les transformées de Laplace des fonctions précédentes en effectuant les substitutions suivantes :

$C_{f1}(p) = f * \Omega_m(\mathrm{p}) + \mathrm{C}_{r1}(p)$

$C_{f2}(p) = f * \Omega_r(\mathrm{p}) + \mathrm{C}_{r2}(p)$

Avec $\mathrm{C}_{r1}(p) = \mathrm{C}_{r2}(p) = C_{r0}(p)$ et $\Omega_r = \frac{1}{x} * \Omega_m$ (principe de réduction)

$\mathrm{C_g} = K_c I_g(p)$

$\mathrm{C_m} = K_c I_m(p)$

On obtient :

$$I_m(p) = \frac{-\frac{Ke}{L}\Omega_m(p) + \frac{1}{L}U(p)}{p + \frac{R}{L}}$$

$$I_g(p) = \frac{\frac{Ke}{L}\Omega_m(p)}{p + \frac{R}{L} + \frac{R_c}{L}}$$

$$\Omega_m(\mathrm{p}) = \frac{K_c I_m(p) - K_c I_g(p) - f\Omega_m(\mathrm{p}) - \mathrm{C}_{r0}(p) - \frac{1}{x^2} f\Omega_m(\mathrm{p}) - \frac{1}{x}\mathrm{C}_{r0}(p)}{Jp\,(p + \frac{R}{L})}$$

En remplaçant $\left(1 + \frac{1}{\mathrm{x}^2}\right)\Omega_m$ par $\tilde{f}\Omega_m$ et en substituant les expressions de $I_m(p)$ et $I_g(p)$:

$$\Omega_m(\mathrm{p})\left(1 - \frac{K_E K_C}{Jp(Lp+R)} - \frac{K_E K_C}{Lp + R + R_C} + \tilde{f}\right) = \frac{1}{Jp(Lp+R)}U(p) - \frac{1}{J}P_r(p)$$

Où $P_r(p) = \frac{\left(1+\frac{1}{x}\right)\mathrm{C}_{r0}(p)}{p}$

D'où :

$$\Omega_m(p) = \frac{(R + R_c + Lp) * K_c}{2 * K_c * K_e(\frac{R_c}{2} + R + Lp) + (\tilde{f} + Jp) * (R + Lp) * (R_c + R + Lp)} * U(p) - \frac{(R + Lp) * (R_c + R + Lp)}{2 * K_c * K_e(\frac{R_c}{2} + R + Lp) + (\tilde{f} + Jp) * (R + Lp) * (R_c + R + Lp)} P_r(p)$$

Avec $\tilde{f} = f * (1 + \frac{1}{x^2})$ et comme le système est linéaire, nous avons des pôles commun et ainsi les dénominateurs sont les mêmes comme nous pouvons le voir.

La génératrice tachymétrique délivre à ses bornes une tension proportionnelle à Ω_m : $\mathrm{U_{GT}} = \mathrm{K_\Omega}\, \Omega_m$

Alors la fonction de transfert entre la tension appliquée, les perturbations statiques et la tension U_{GT} est donnée par :

$$U_{GT}(p) = \frac{(R + R_c + Lp) * K_c\, K_\Omega}{2 * K_c * K_e(\frac{R_c}{2} + R + L_p) + (\tilde{f} + Jp) * (R + Lp) * (R_c + R + Lp)} * U(p) - \frac{K_\Omega\, (R + Lp) * (R_c + R + Lp)}{2 * K_c * K_e(\frac{R_c}{2} + R + Lp) + (\tilde{f} + Jp) * (R + Lp) * (R_c + R + Lp)} P_r(p)$$

La tension recueillie par le potentiomètre est proportionnelle à l'angle Θ_r qui est égal à l'intégrale de Ω_r.

En appliquant le principe de réduction on obtient :

$$U_p(p) = \mathrm{K_\Theta}\Theta_r = \mathrm{K_\Theta}\frac{\Omega_\mathrm{r}}{p} = \frac{\mathrm{K_\Theta}\, \Omega_m}{xp}$$

$$U_p(p) = \frac{(R + R_c + Lp) * K_c\, K_\Theta}{2xpK_c * K_e(\frac{R_c}{2} + R + Lp) + 2xp(\tilde{f} + Jp) * (R + Lp) * (R_c + R + Lp)} * U(p) - \frac{K_\Theta\, (R + Lp) * (R_c + R + Lp)}{4xpK_c * K_e(\frac{R_c}{2} + R + Lp) + 2xp(\tilde{f} + Jp) * (R + Lp) * (R_c + R + Lp)} P_r(p)$$

1.3.4 On suppose que le frottement visqueux est négligeable ($f \approx 0$)

$$U_{GT}(p) = \frac{(R + R_c + Lp) * K_c\, K_\Omega}{2 * K_c * K_e\left(\frac{R_c}{2} + R + Lp\right) + (Jp) * (R + Lp) * (R_c + R + Lp)} * U(p) - \frac{K_\Omega\, (R + Lp) * (R_c + R + Lp)}{2 * K_c * K_e\left(\frac{R_c}{2} + R + Lp\right) + (Jp) * (R + Lp) * (R_c + R + Lp)} P_r(p)$$

$$U_p(p) = \frac{(R + R_c + Lp) * K_c\, K_\Theta}{2xpK_c * K_e(\frac{R_c}{2} + R + Lp) + 2xp(Jp) * (R + Lp) * (R_c + R + Lp)} * U(p) - \frac{K_\Theta\, (R + Lp) * (R_c + R + Lp)}{4xpK_c * K_e\left(\frac{R_c}{2} + R + Lp\right) + 2xp(Jp) * (R + Lp) * (R_c + R + Lp)} P_r(p)$$

1.3.5 Faire de même en supposant $R_c = 0$ et $R_c \to +\infty$

Lorsque $R_C = 0$:

$$U_{GT}(p) = \frac{(R+Lp) * K_c\, K_\Omega}{2K_cK_e(R+Lp)+(Jp)*(R+Lp)^2} * U(p) - \frac{K_\Omega\,(R+Lp)^2}{2K_cK_e(R+Lp)+(Jp)*(R+Lp)^2}\, P_r(p)$$

$$U_p(p) = \frac{(R+Lp) * K_c\, K_\Theta}{2xpK_c * K_e(R+Lp)+2Jxp^2*(R+Lp)^2} * U(p) - \frac{K_\Theta\,(R+Lp)^2}{4xpK_c * K_e(R+Lp)+2Jxp^2*(R+Lp)^2}\, P_r(p)$$

Lorsque R_C tend vers $+\infty$ on peut factoriser le numérateur et le dénominateur par R_C :

$$U_{GT}(p) = \frac{\frac{1}{R_c}\left(1+\frac{R}{Rc}+\frac{L}{R_c}p\right) * K_c\, K_\Omega}{\frac{1}{R_c}\left(2K_cK_e\left(\frac{1}{2}+\frac{R}{R_c}+\frac{Lp}{R_c}\right)\right)+\frac{1}{R_c}\left(Jp*(R+Lp)*\left(1+\frac{R}{R_c}+\frac{L}{R_c}p\right)\right)} * U(p) - \frac{\frac{1}{R_c}K_\Omega\,(R+Lp)*(1+\frac{R}{R_c}+\frac{L}{R_c}p)}{\frac{1}{R_c}\left(2K_cK_e\left(\frac{1}{2}+\frac{R}{R_c}+\frac{L}{R_c}p\right)\right)+\frac{1}{R_c}\left(Jp*(R+Lp)*\left(1+\frac{R}{R_c}+\frac{L}{R_c}p\right)\right)}\, P_r(p)$$

Les termes R/R_C et L/R_C tendent vers 0 lorsque R_C tend vers l'infini, d'où :

$$U_{GT}(p) = \frac{K_c\, K_\Omega}{K_cK_e+JRp+JLp^2} * U(p) - \frac{K_\Omega\,(R+Lp)}{K_cK_e+JRp+JLp^2}\, P_r(p)$$

$$U_p(p) = \frac{K_c\, K_\Theta}{K_cK_exp+JRxp^2+JLxp^3} * U(p) - \frac{K_\Theta\,(R+Lp)}{K_cK_exp+JRxp^2+JLxp^3}\, P_r(p)$$

1.4 Propriétés dynamiques

Si l'on a $f = 0$ et $R_c \to +\infty$, alors on trouve :

$$U_{GT}(p) = \frac{\frac{K_\Omega * K_c}{J*L*p_{elec}*p_{meca}}}{(1-\frac{p}{p_{elec}})*(1-\frac{p}{p_{meca}})} U(p) - \frac{(R+L_p)*\frac{K_\Omega}{J*L*p_{elec}*p_{meca}}}{(1-\frac{p}{p_{elec}})*(1-\frac{p}{p_{meca}})} P_\Omega(p)$$

Avec :

$p_{elec} = -\frac{R}{2*L} * (1 + \sqrt{1 - \frac{4*L*K_c*K_e}{J*R^2}})$ et $p_{méca} = -\frac{R}{2*L} * (1 - \sqrt{1 - \frac{4*L*K_c*K_e}{J*R^2}})$ et

- $K_c = 16.3\, m\, Nm/A$ soit $K_c = 1.63 * 10^{-2}\, N.m.A^{-1}$
- $K_e = \frac{1}{585}\, tr^{-1}/min/V$ soit $K_e = \frac{30}{\pi*585}\, V.s.rad^{-1}$
- $R = 1.34\, \Omega$
- $L = 0.12\, mH$ soit $L = 1.2 * 10^{-4}\, H$
- $J = 2 * J_m = 2 * 9.19\, g * cm^2$ soit $J = 2 * 9.19 * 10^{-7}\, kg * m^2$

Après conversion en USI

On a donc : $p_{elec} = -\frac{R}{2*L} * (1 + \sqrt{1 - \frac{4*L*K_c*K_e}{J*R^2}}) = -11\,057.56868 \approx -11\,000$

$$p_{méca} = -\frac{R}{2*L} * (1 - \sqrt{1 - \frac{4*L*K_c*K_e}{J*R^2}}) \approx -109.097983 \approx -110$$

Et $p_{elec} * p_{méca} = \frac{K_c*K_e}{J*L}$ après simplification

Finalement, nous obtenons : $U_{GT}(p) \approx \frac{K_v}{1+\tau*p} * U(p) - \frac{\widetilde{K_v}}{1+\tau*p} * P_r(p)$

Où $K_v = \frac{K_\Omega}{K_e}$, $\widetilde{K_v} = K_v * \frac{R}{K_c}$ et $\tau = -\frac{1}{p_{méca}}$

Nous remarquons également que la structure du modèle du servo-mécanisme est une structure bruitée de type ARMAX. Cela est presque une structure ARX à une constante près.

2. Simulations numériques

2.1 Saisie des différents paramètres et des données numériques

Voici les différents paramètres et données numériques que nous avons récupéré depuis l'énoncé ainsi que les calculs précédents. Nous avons essayé au maximum de convertir les grandeurs que nous allons utiliser en USI :

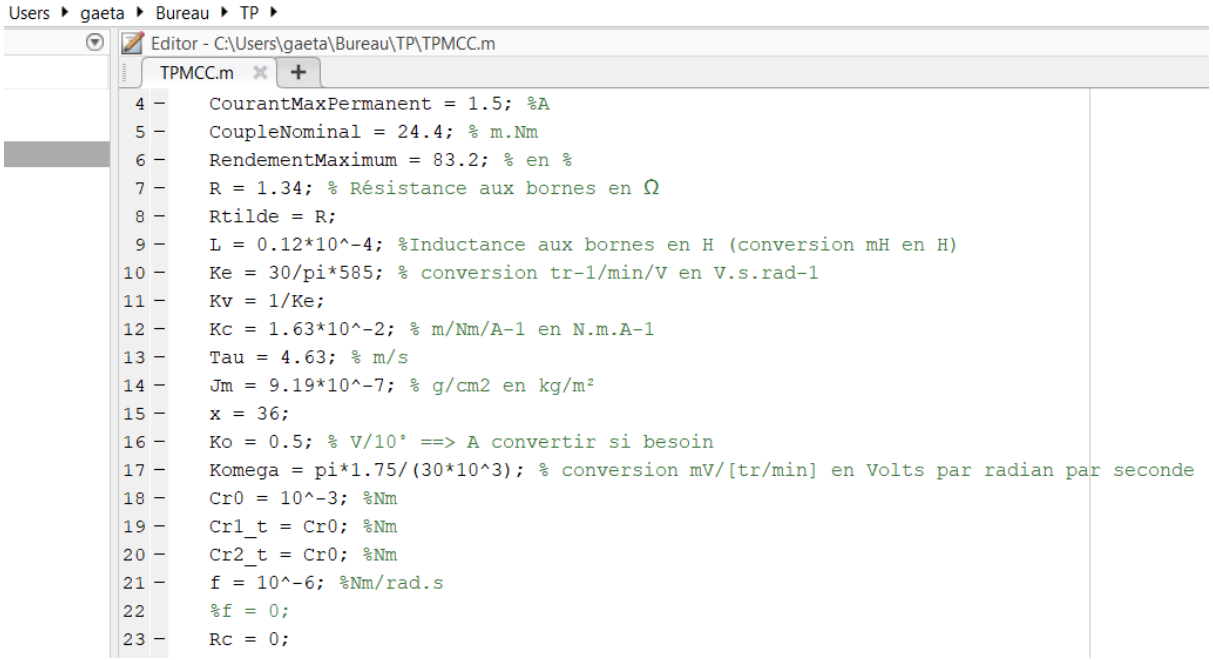


Figure 3 : Code Matlab avec les différents paramètres et les données numériques

2.2 Construire le schéma-bloc Simulink représentant le modèle complet du servomécanisme

Voici le modèle complet du servomécanisme d'après le schéma bloc que nous avons réalisé à partir de la partie 1.2 Schéma-bloc :

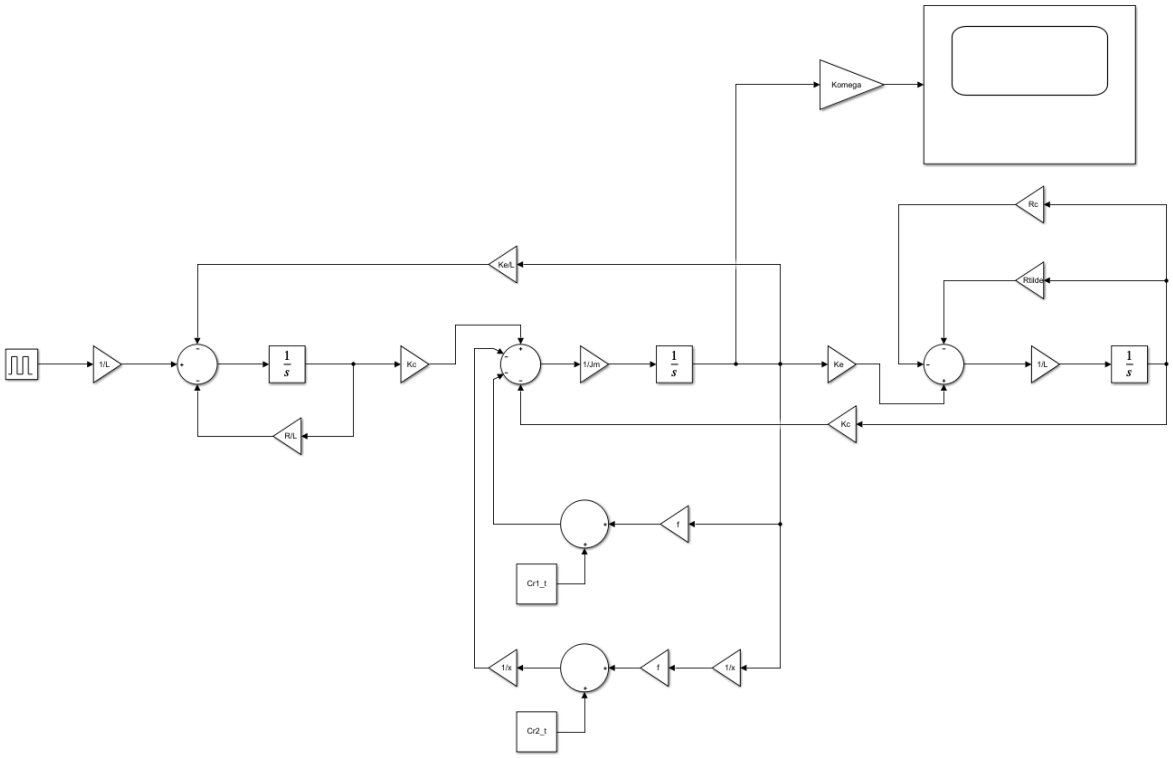


Figure 4 : Schéma-bloc Simulink

2.3 Pour $f = 0$ et $R_c = 0$, étudier l'influence de C_{r0} sur le temps de réponse et le gain statique

Le temps de réponse d'un signal fait référence au temps nécessaire à un système pour répondre à un changement dans le signal d'entrée. Plus précisément, le temps de réponse d'un signal mesure le temps écoulé entre le moment où le signal d'entrée change et le moment où le système produit un changement correspondant dans le signal de sortie.

Le temps de réponse total est le temps nécessaire pour que le système passe de 10 % à 90 % de sa valeur finale après un changement de signal d'entrée.

Le gain statique est le rapport de la variation de la sortie finale sur la variation d'entrée (échelon) une fois le système stabilisé avec comme formule : $k = \frac{\Delta s}{\Delta e}$

Ce qui est important à dire est que nous avons donc défini un signal d'entrée qui est un échelon de tension d'amplitude U_0 = 10V avec une période de 0.1 seconde (pour des raisons de performance) et et un rapport cyclique de 50% :

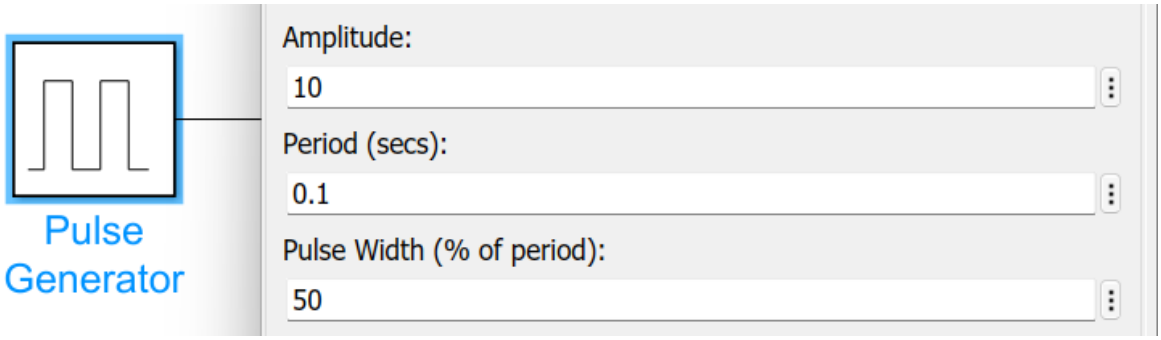


Figure 5 : Choix des caractéristiques du signal d'entrée

Voici ce que nous obtenons lors d'une simulation pendant 0.5 seconde :

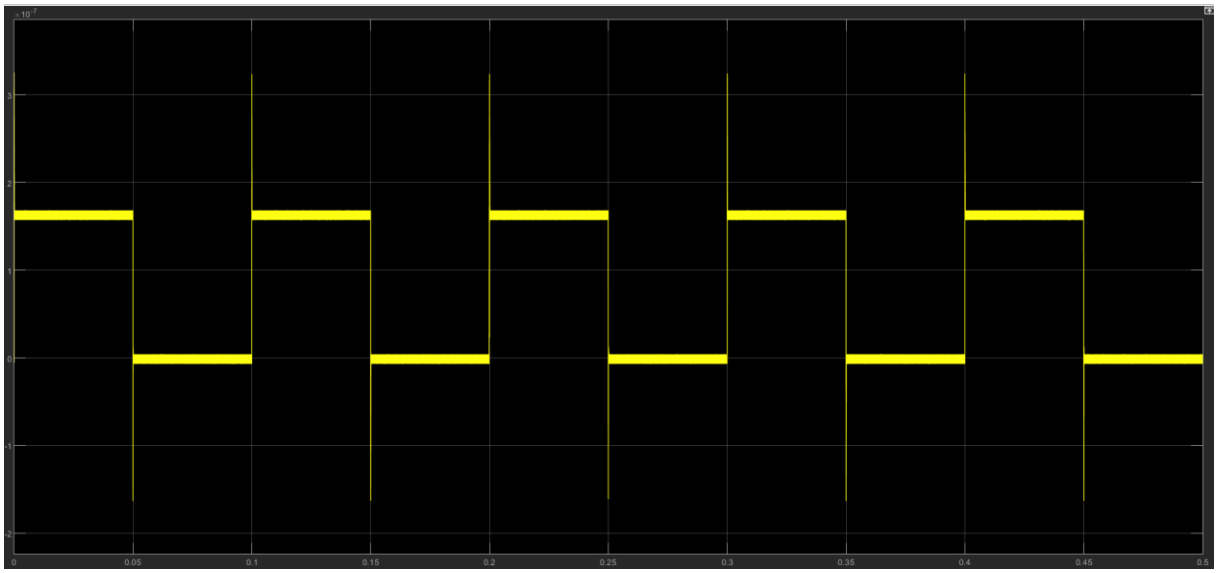


Figure 6 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec

Si nous zoomons sur les transitions entre temps bas et temps haut de notre signal, nous remarquons des oscillations :

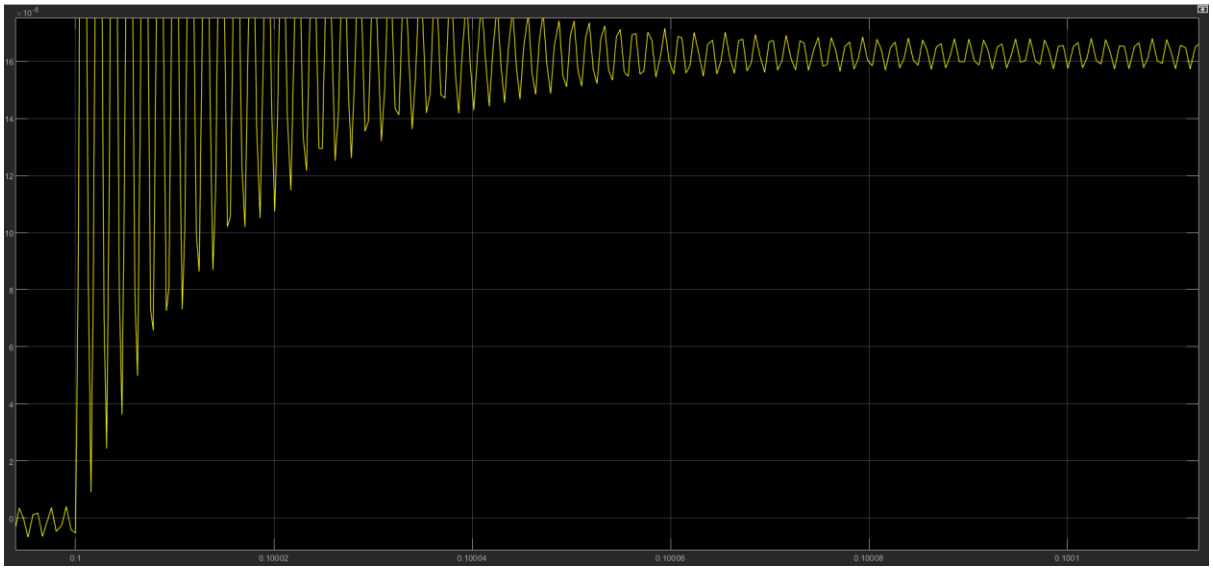


Figure 7 : Oscillations visibles avant stabilisation du signal

Ainsi, nous pouvons facilement déterminer le temps de réponse et le gain statique à partir du signal de sortie :

$\text{Temps de réponse} = 0{,}00004\ \text{sec} = 0.04\ \text{ms}$ soit $\frac{4}{10000} = \frac{1}{2500}$ temps de la période

$$\text{Gain statique} = \text{k} = \frac{\Delta s}{\Delta e} = \frac{1.63 * 10^{-7}}{10} = 1.63 * 10^{-8}$$

Lorsque nous faisons varier le paramètre C_{r0} de plus 50%, voici ce que nous obtenons :

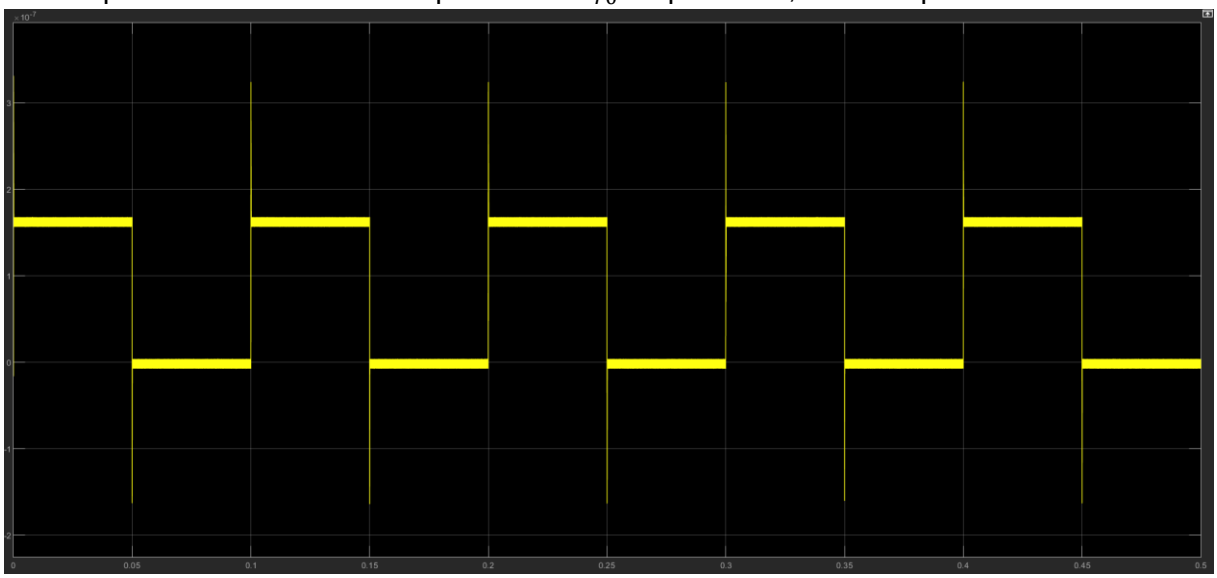


Figure 8 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en faisant varier le paramètre C_{r0} de plus 50%

Nous remarquons qu'augmenter la valeur du paramètre C_{r0} diminue légèrement le gain statique et augmente assez significativement les oscillations.

Lorsque nous faisons varier le paramètre C_{r0} de moins 50%, voici ce que nous obtenons :

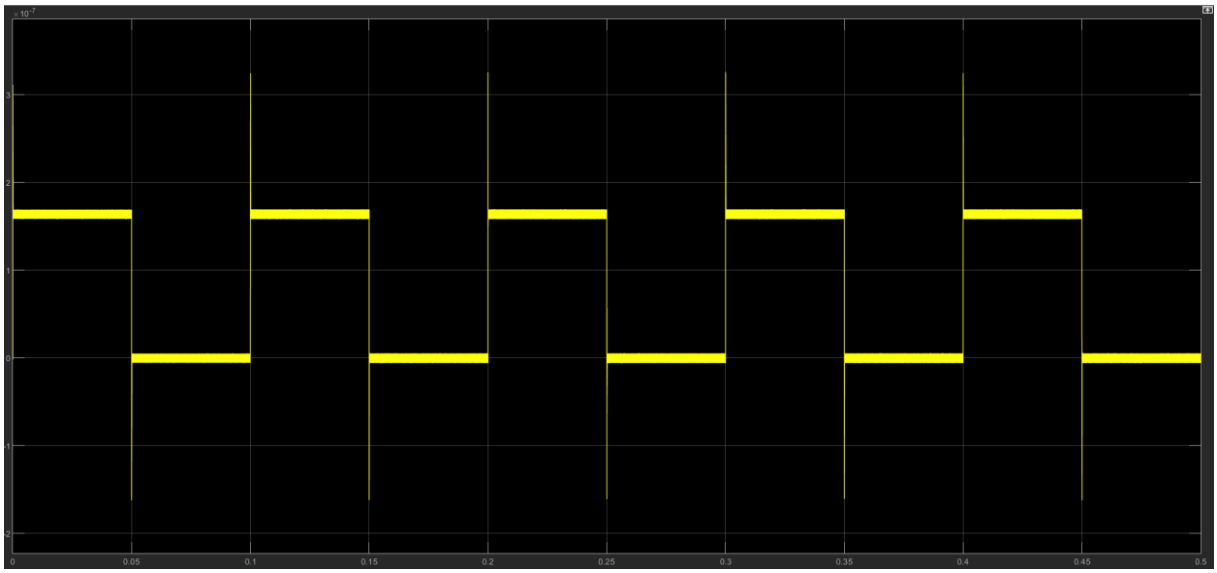


Figure 9 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en faisant varier le paramètre C_{r0} de moins 50%

Nous remarquons que lorsque nous augmentons la valeur du paramètre C_{r0}, cela diminue le gain statique ainsi que le temps de réponse mais augmente les oscillations.

2.4 Étudier l'influence de f et R_c de sur le temps de réponse et le gain statique

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 0$, voici ce que nous obtenons :

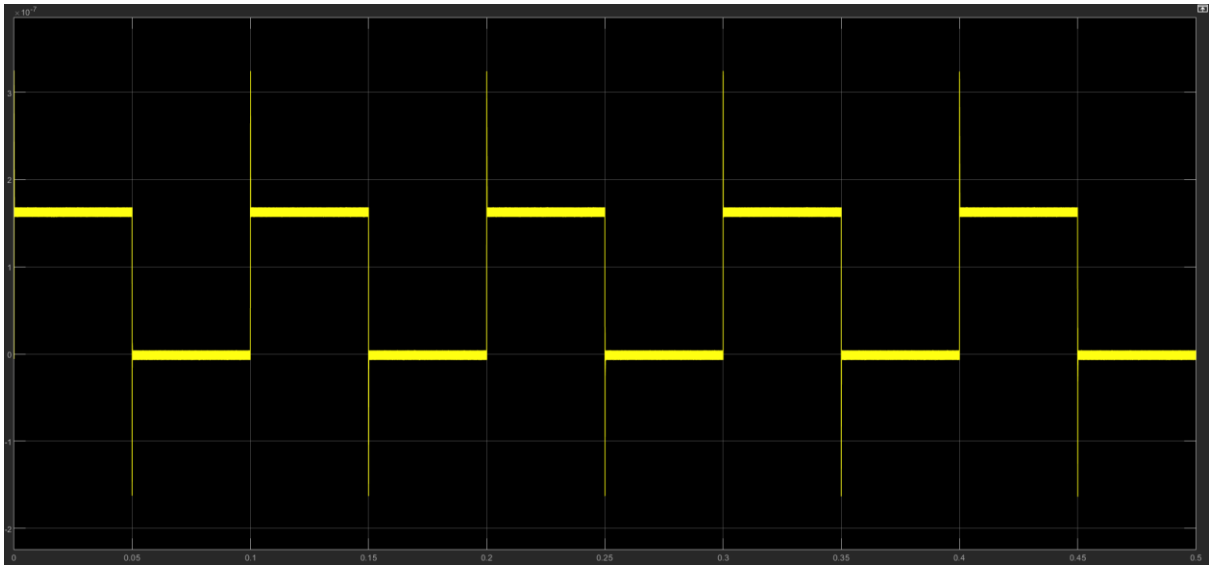


Figure 10 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant le paramètre $f = 10^{-6}$

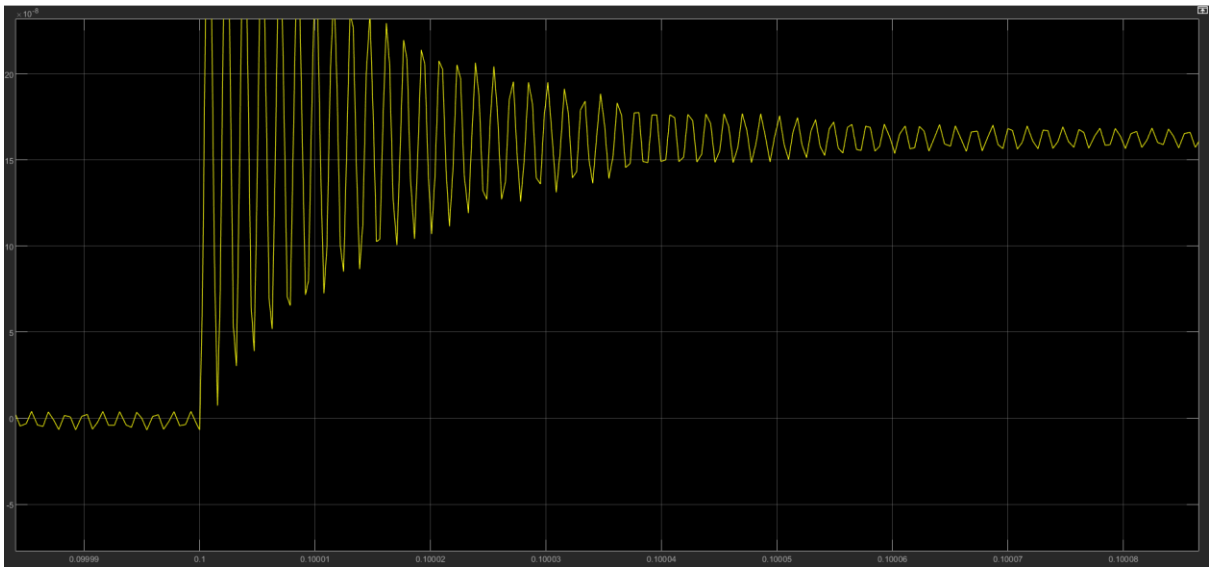


Figure 11 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant le paramètre $f = 10^{-6}$

Lorsque $\boldsymbol{f = 2 * 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 0$, voici ce que nous obtenons :

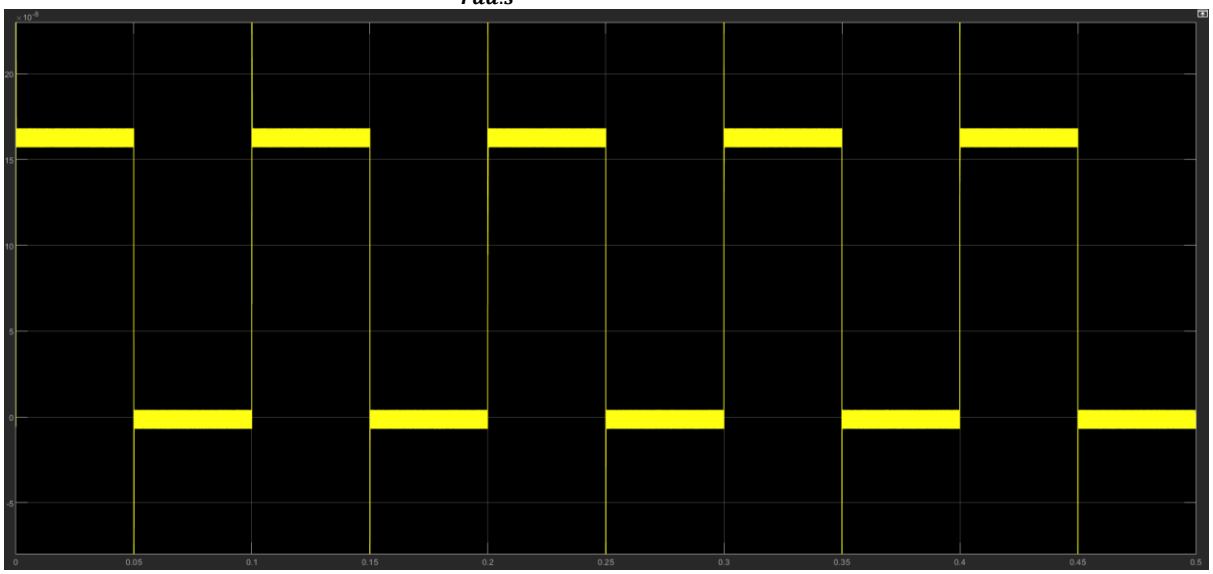


Figure 12 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant le paramètre $f = 2 * 10^{-6}$

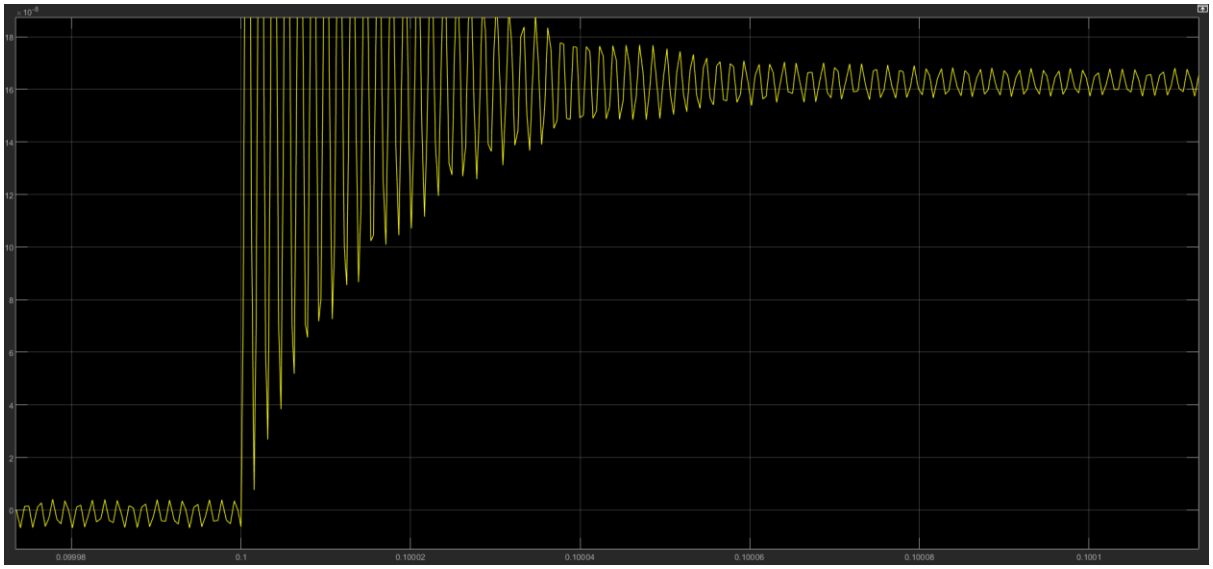


Figure 13 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant le paramètre $f = 2 * 10^{-6}$

Lorsque $\boldsymbol{f = 0.5 * 10^{-6}}$ $\frac{\boldsymbol{Nm}}{\boldsymbol{rad.s^1}}$ et $\boldsymbol{R_c} = 0$, voici ce que nous obtenons :

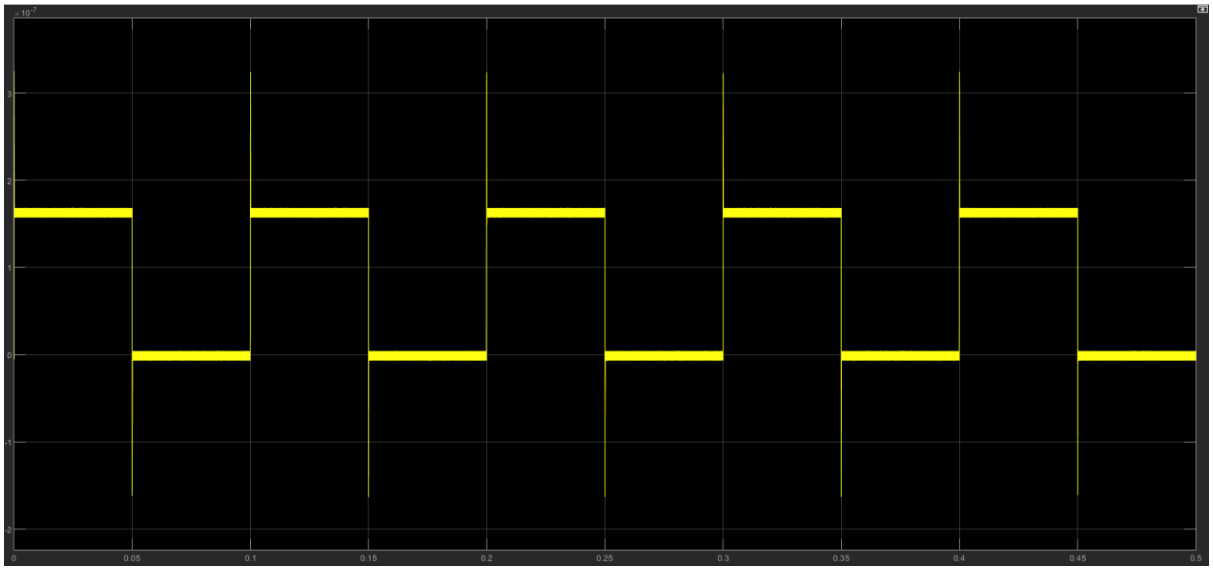


Figure 14 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant le paramètre $f = 0.5 * 10^{-6}$

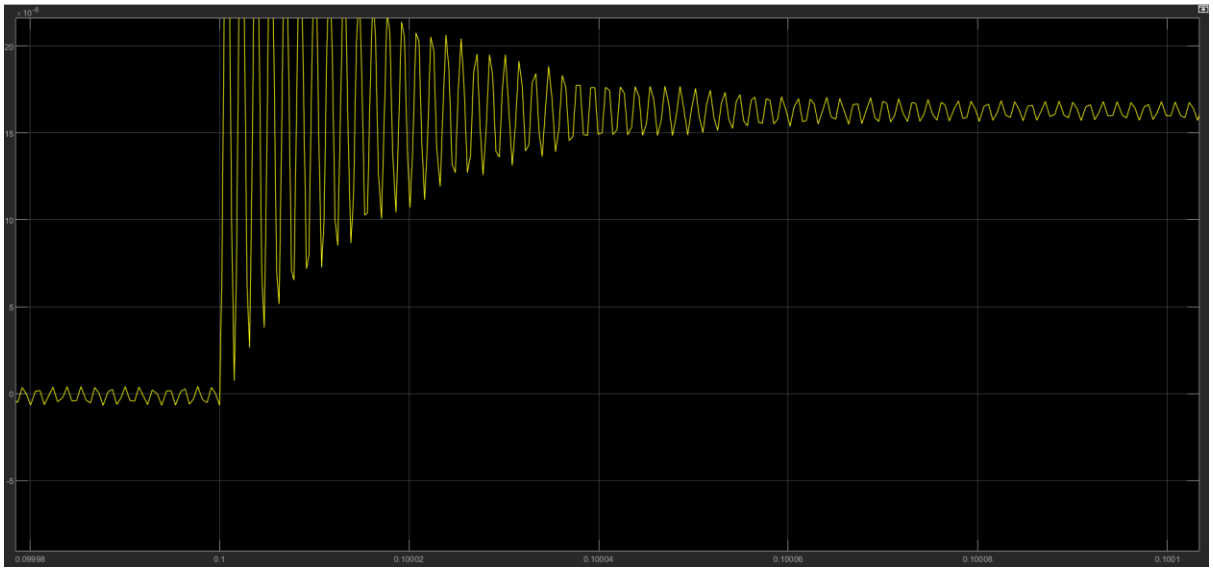


Figure 15 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant le paramètre $f = 0.5 * 10^{-6}$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 1$, voici ce que nous obtenons :

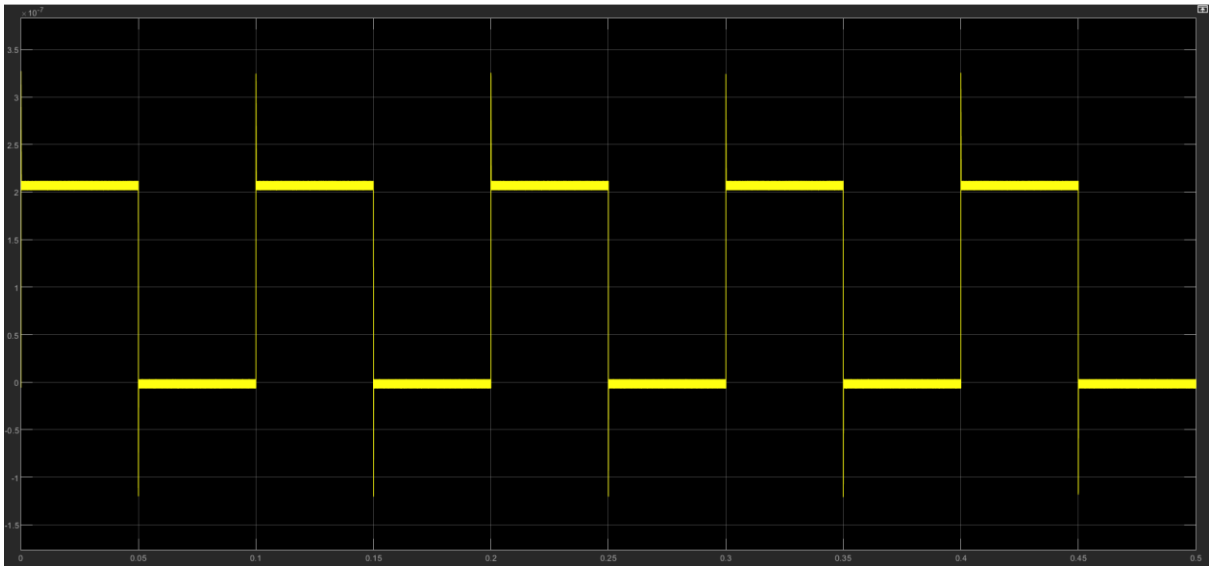


Figure 16 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 1$

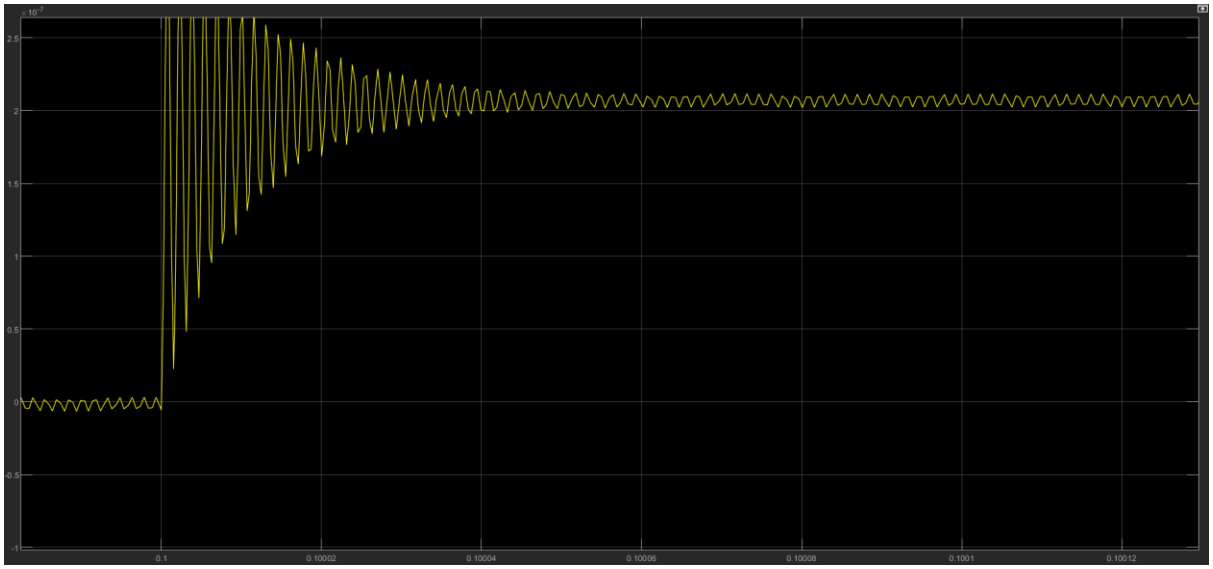


Figure 17 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 1$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 10$, voici ce que nous obtenons :

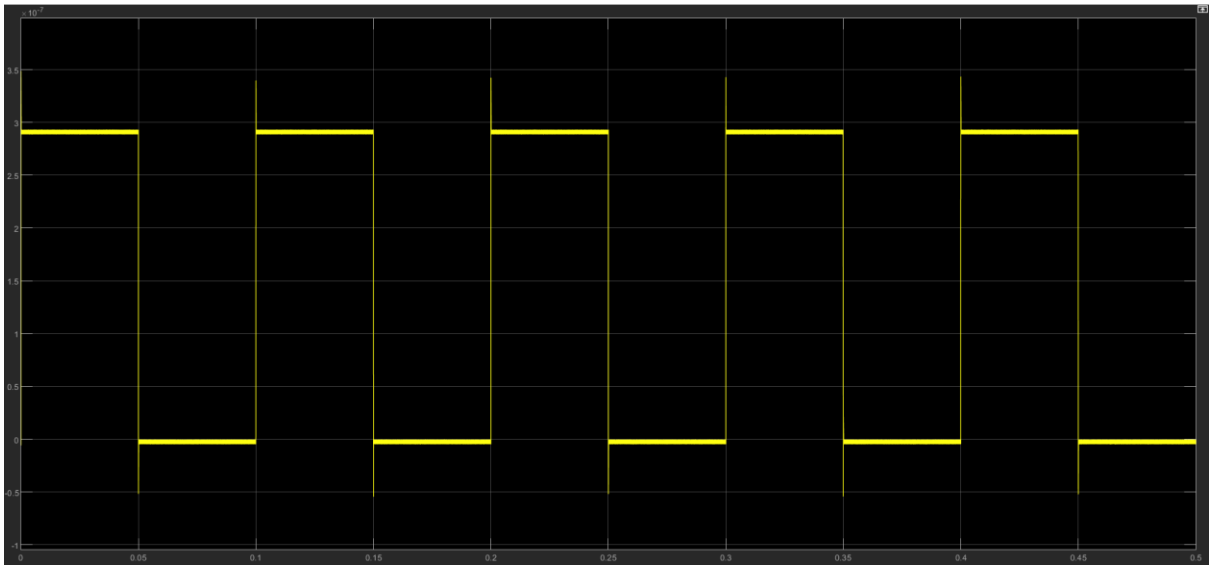


Figure 18 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 10$

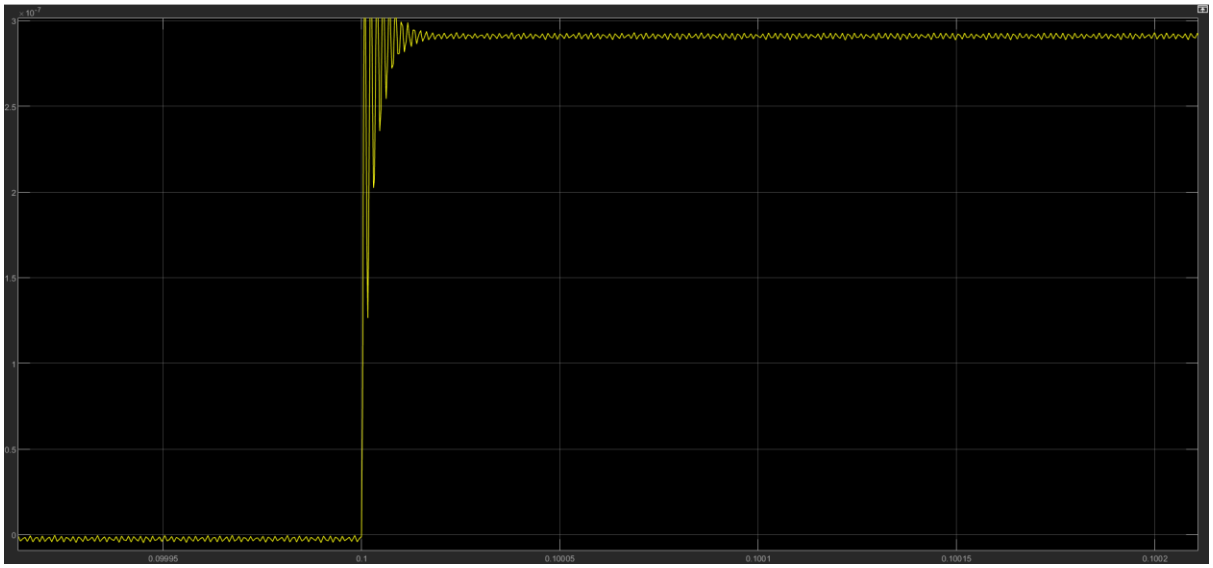


Figure 19 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 10$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 100$, voici ce que nous obtenons :

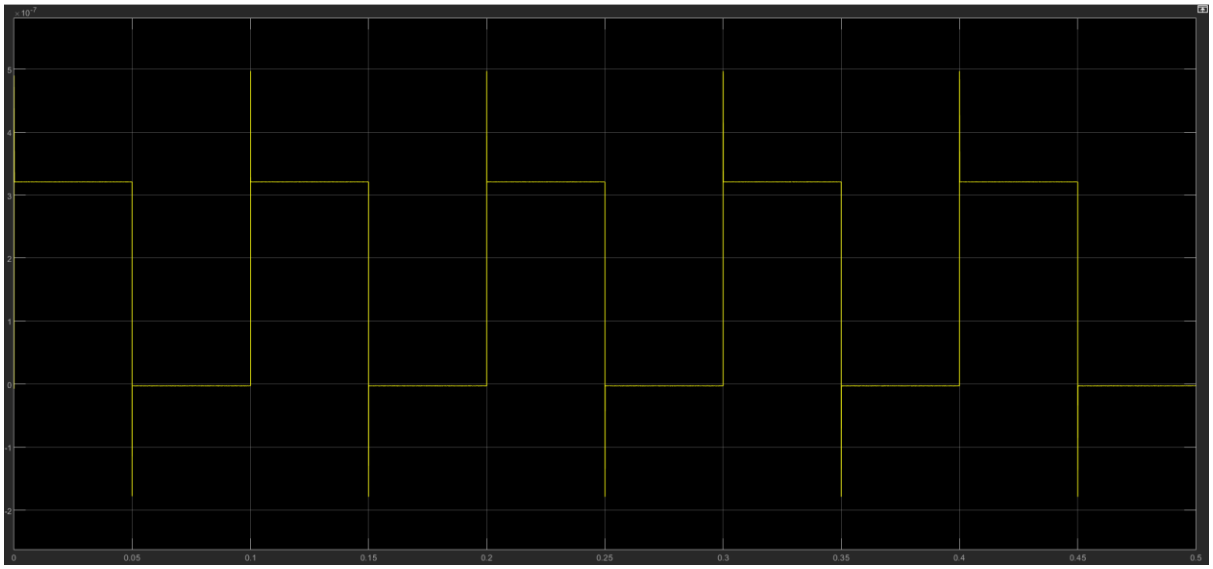


Figure 20 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 100$

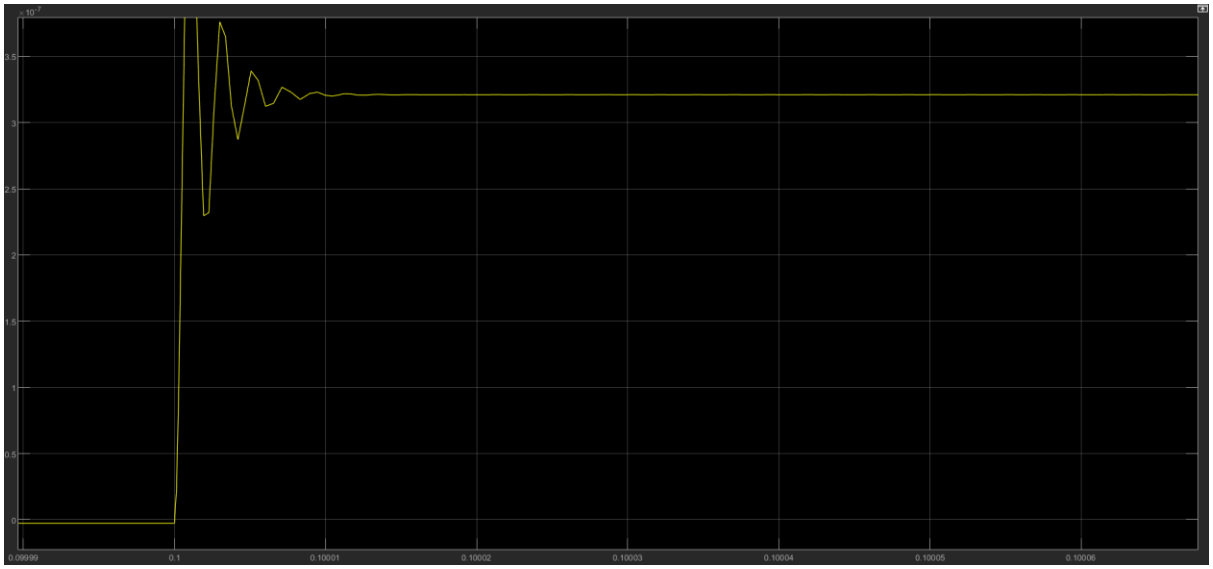


Figure 21 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 100$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 1000$, voici ce que nous obtenons :

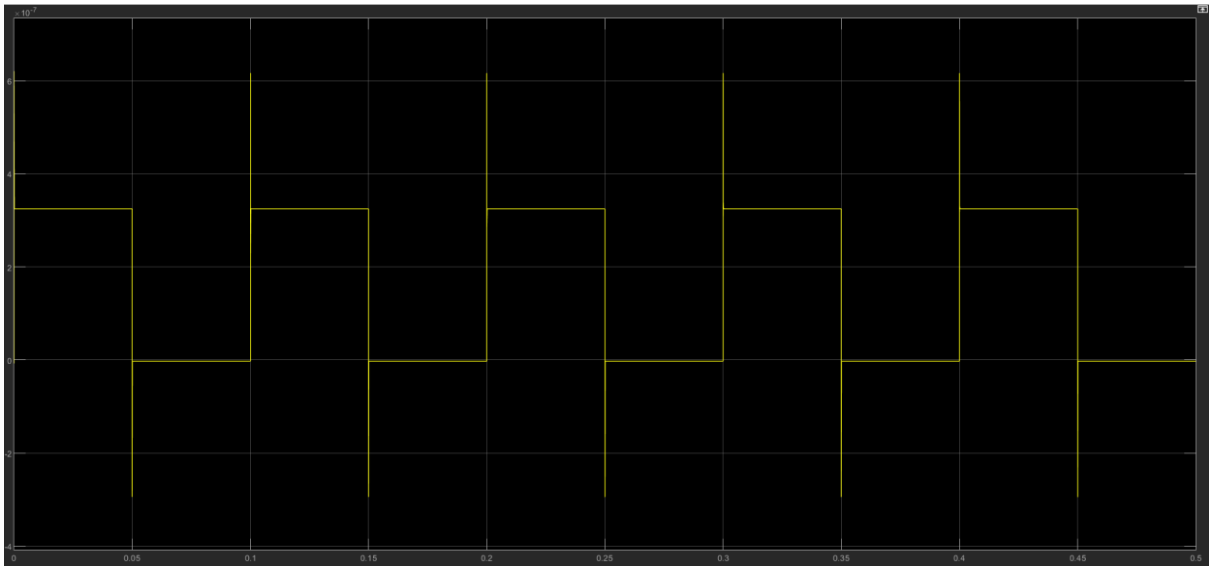


Figure 22 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 1000$

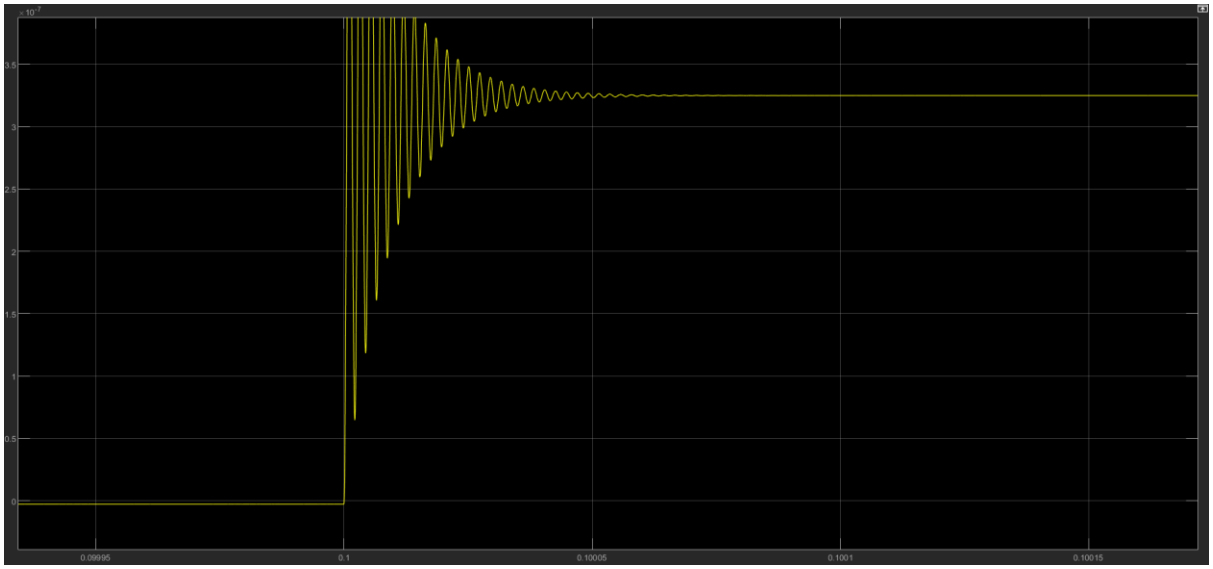


Figure 23 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 1000$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 10000$, voici ce que nous obtenons :

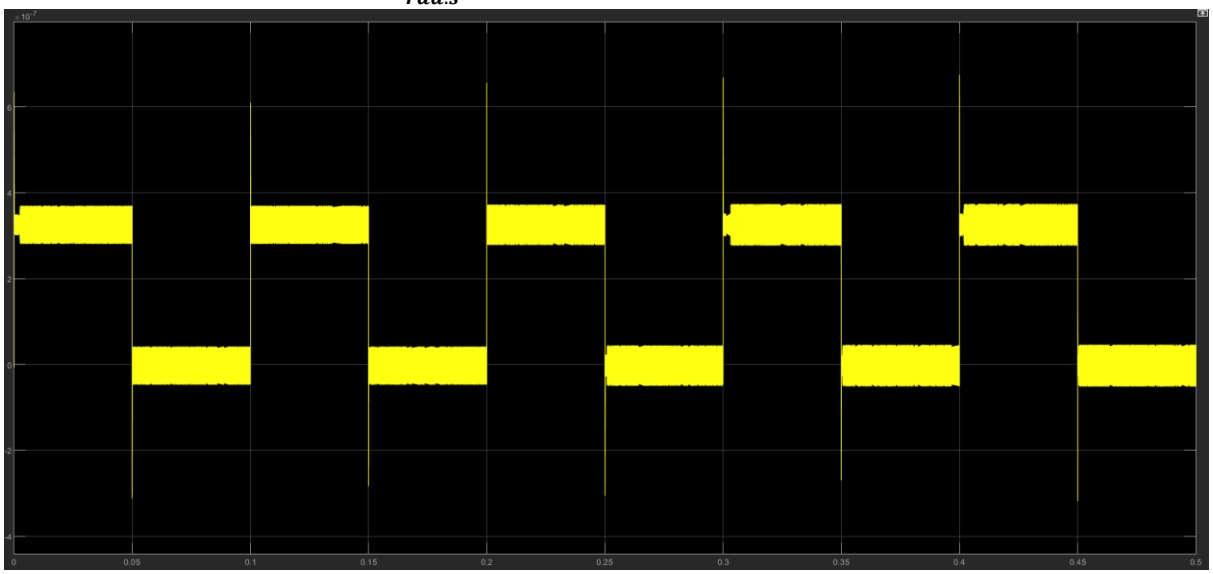


Figure 24 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 10000$

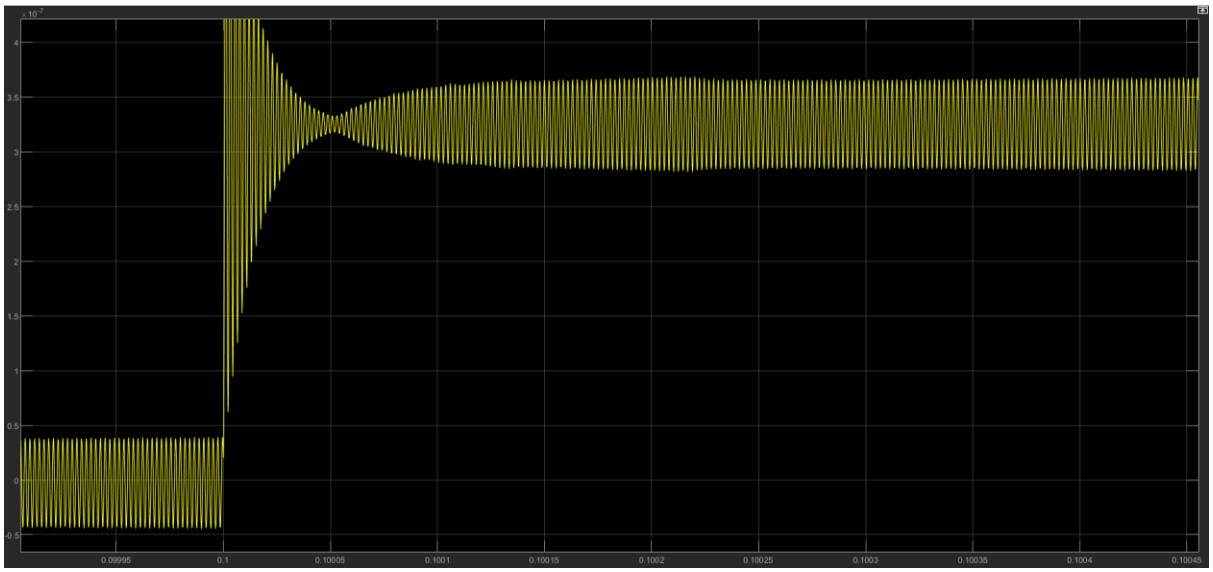


Figure 25 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 10000$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 100000$, voici ce que nous obtenons :

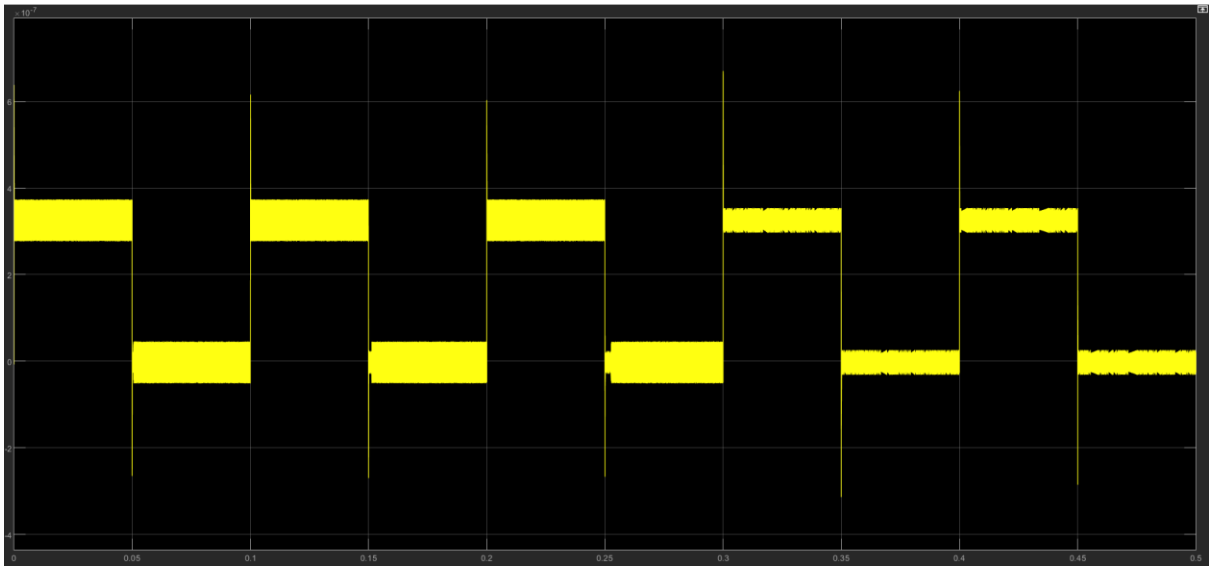


Figure 26 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 100000$

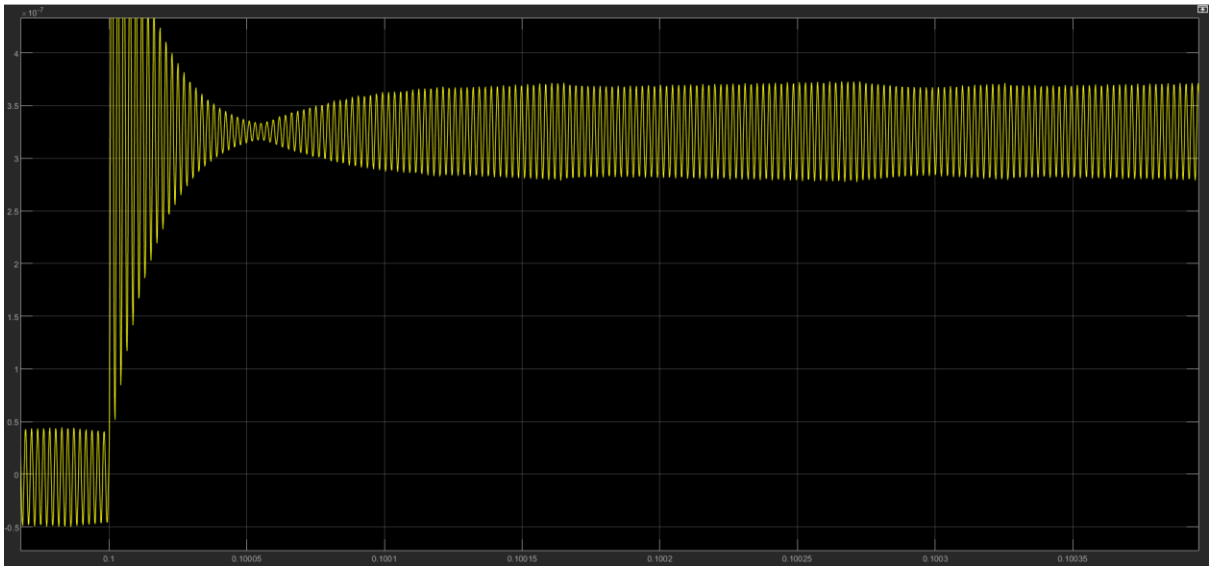


Figure 27 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 100000$

Lorsque $\boldsymbol{f = 10^{-6}\ \frac{Nm}{rad.s^1}}$ et $\boldsymbol{R_c} = 1000000$, voici ce que nous obtenons :

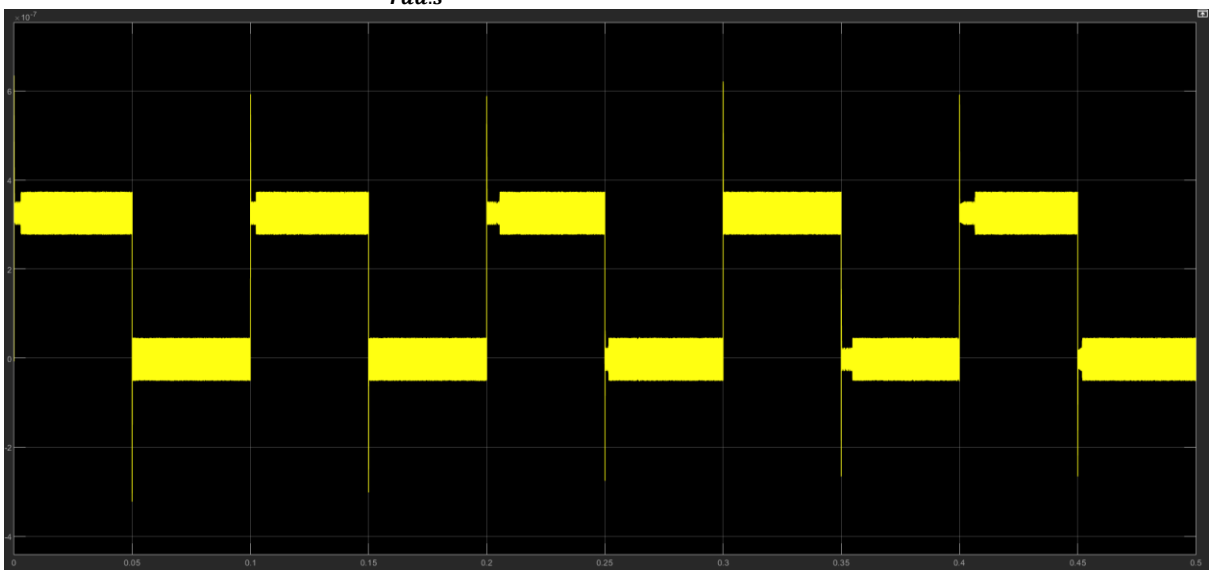


Figure 28 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 1000000$



Figure 29 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 1000000$

Lorsque $\boldsymbol{f = 10^{-6}}$ $\frac{\boldsymbol{Nm}}{\boldsymbol{rad.s^1}}$ et $\boldsymbol{R_c} = 10000000$, voici ce que nous obtenons :

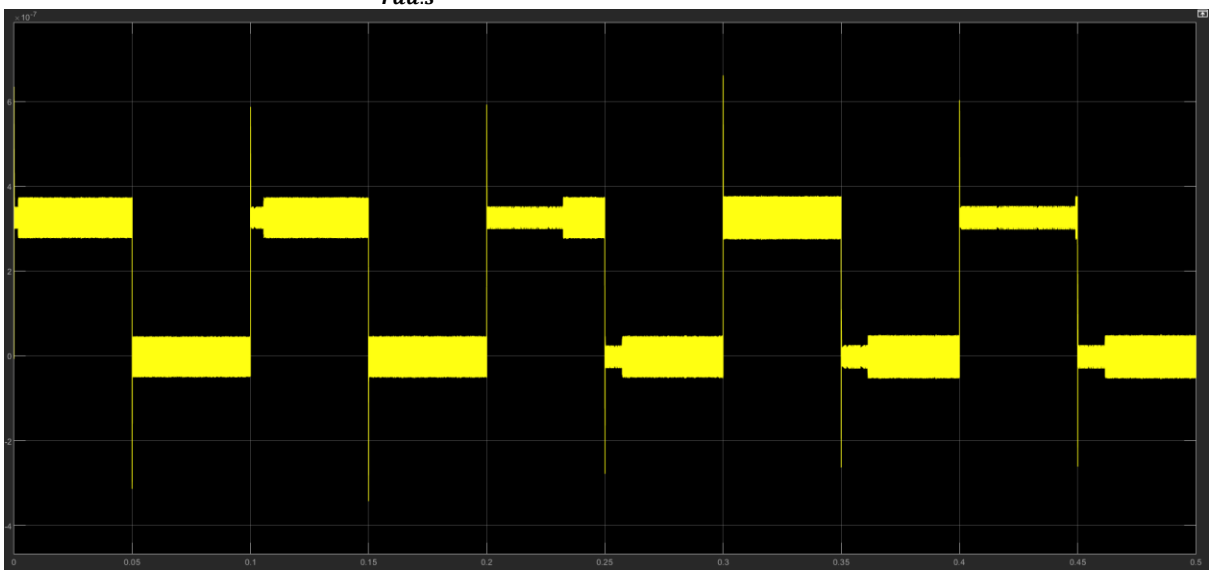


Figure 30 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 10000000$

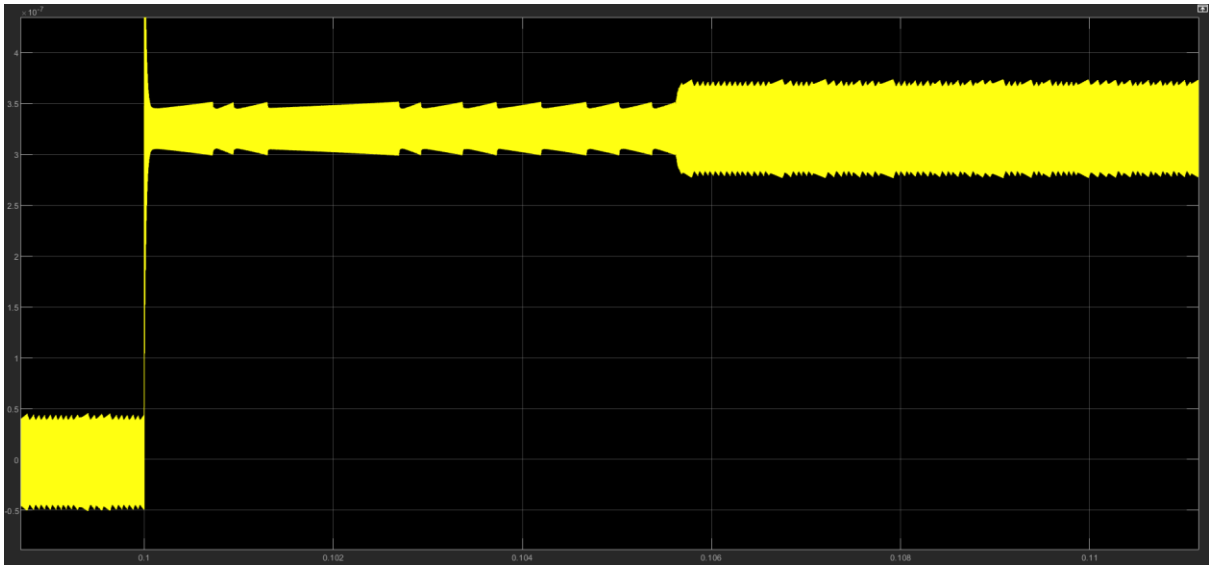


Figure 31 : Signal de sortie pour une simulation de 0.5 sec avec une période du signal d'entrée de 0.1 sec en définissant les paramètres $f = 10^{-6}$ et $R_c = 10000000$

Concernant les frottements $\boldsymbol{f}$, il semble que lorsque ces derniers augmentent, les perturbations semblent également augmenter ce qui semble logique. Cependant, les frottements ont un impact modéré sur le gain statique (bien qu'ils devraient l'atténuer) ainsi que sur le temps de réponse (qu'ils devraient augmenter).

Comme nous pouvons le voir, augmenter la valeur de $\mathbf{R_c}$ augmente la valeur du gain statique d'environ $2.1 * 10^{-7}$ avec $\mathbf{R_c} = \mathbf{1}$ jusqu'à $3.25 * 10^{-7}$ pour $\mathbf{R_c} = \mathbf{100000000}$ soit une augmentation d'environ 50%. Nous remarquons également qu'augmenter la valeur de $\mathrm{R_c}$ augmente fortement les oscillations ainsi que le temps de réponse (sauf pour les valeurs où $\mathbf{R_c} = 100$ et $\mathbf{R_c} = 1000$, les résultats sont très bons).

2.5 Calcul des pôles et évolution dans le plan complexe

Nous savons que :

$$\mathrm{FT}_{U/U_{GT}}(p) = \frac{K_\Omega * K_c}{J * L * p^2 + J * R * p + K_e * K_c}$$

Que nous pouvons ainsi traduire en code Matlab :

```
% Définir la fonction de transfert
num = [Komega*Kc]; % Coefficients du numérateur
den = [J*L, J*R, Ke*Kc]; % Coefficients du dénominateur
sys = tf(num, den) % Créer l'objet fonction de transfert

% Calculer les pôles de la fonction de transfert
p = pole(sys); % Trouver les pôles

% Afficher les pôles
disp('Les pôles de la fonction de transfert sont :')
disp(p)

zplane(num, den); % Tracer les pôles
title('Evolution des pôles dans le plan complexe');
```

Figure 32 : Calcul des pôles de la fonction de transfert de U/U_{GT}

Ainsi, nous obtenons les pôles suivants :

```
                2.987e-06
  -----------------------------------
  2.206e-11 s^2 + 2.463e-06 s + 91.06

Continuous-time transfer function.

Les pôles de la fonction de transfert sont :
   1.0e+06 *

  -0.0558 + 2.0311i
  -0.0558 - 2.0311i
```

Figure 33 : Calcul des pôles de la fonction de transfert de U/U_{GT}

On a donc $p_1 = 10^6 * (-0.0558 + 2.0311 * i)$ et $p_2 = 10^6 * (-0.0558 - 2.0311 * i)$:

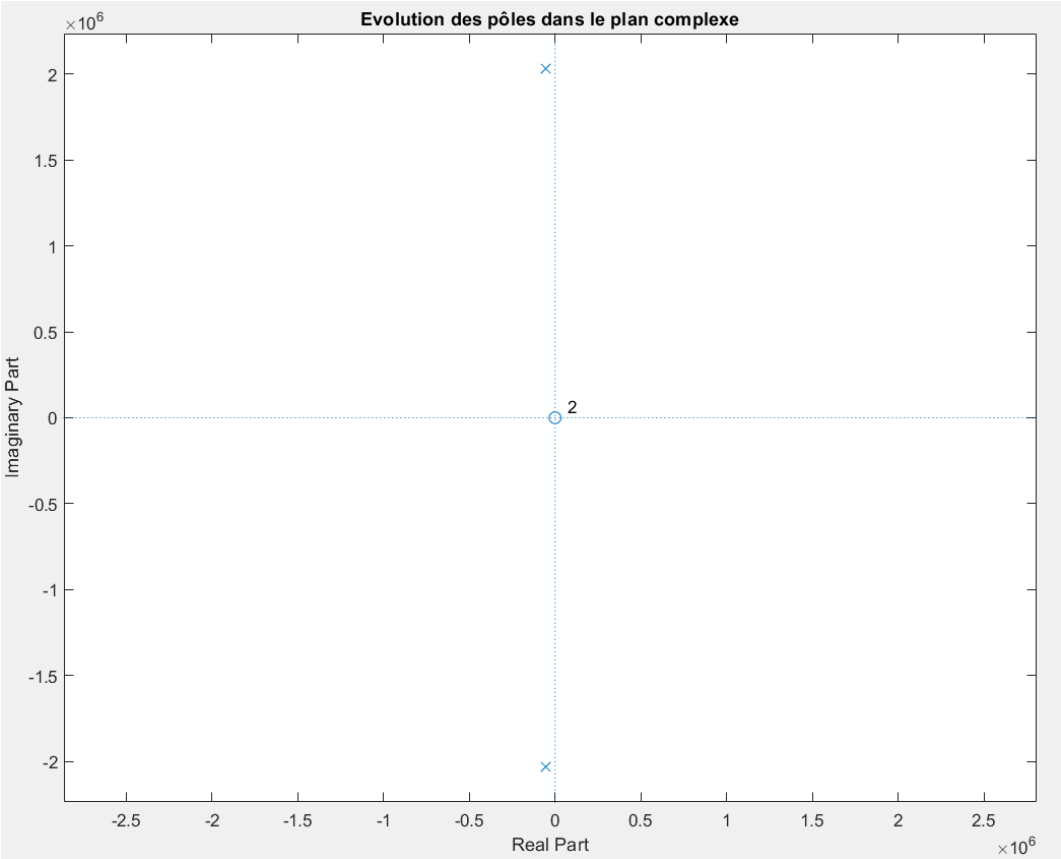


Figure 34 : Évolution des pôles dans le plan complexe

Nous savons que :

$$\mathrm{FT}_{U/U_p}(p) = \frac{K_\theta * K_c}{J * L * x * p^3 + J * R * x * p^2 + x * K_e * K_c * p}$$

Que nous pouvons ainsi traduire en code Matlab :

```
% Définir la fonction de transfert
num = [Ko*Kc]; % Coefficients du numérateur
den = [J*L*x, J*R*x, x*Ke*Kc, 0]; % Coefficients du dénominateur
sys = tf(num, den) % Créer l'objet fonction de transfert

% Calculer les pôles de la fonction de transfert
p = pole(sys); % Trouver les pôles

% Afficher les pôles
disp('Les pôles de la fonction de transfert sont :')
disp(p)

zplane(num, den); % Tracer les pôles
title('Evolution des pôles dans le plan complexe');
```

Figure 35 : Calcul des pôles de la fonction de transfert de U/U_p

Ainsi, nous obtenons les pôles suivants :

```
                  0.0467
-------------------------------------
7.94e-10 s^3 + 8.867e-05 s^2 + 3278 s

Continuous-time transfer function.

Les pôles de la fonction de transfert sont :
   1.0e+06 *

   0.0000 + 0.0000i
  -0.0558 + 2.0311i
  -0.0558 - 2.0311i
```

Figure 36 : Calcul des pôles de la fonction de transfert de U/U_p

On a donc $p_1 = 10^6 * (0.0000 + 0.0000 * i)$ et $p_2 = 10^6 * (-0.0558 + 2.0311 * i)$ et $p_3 = 10^6 * (-0.0558 - 2.0311 * i)$:

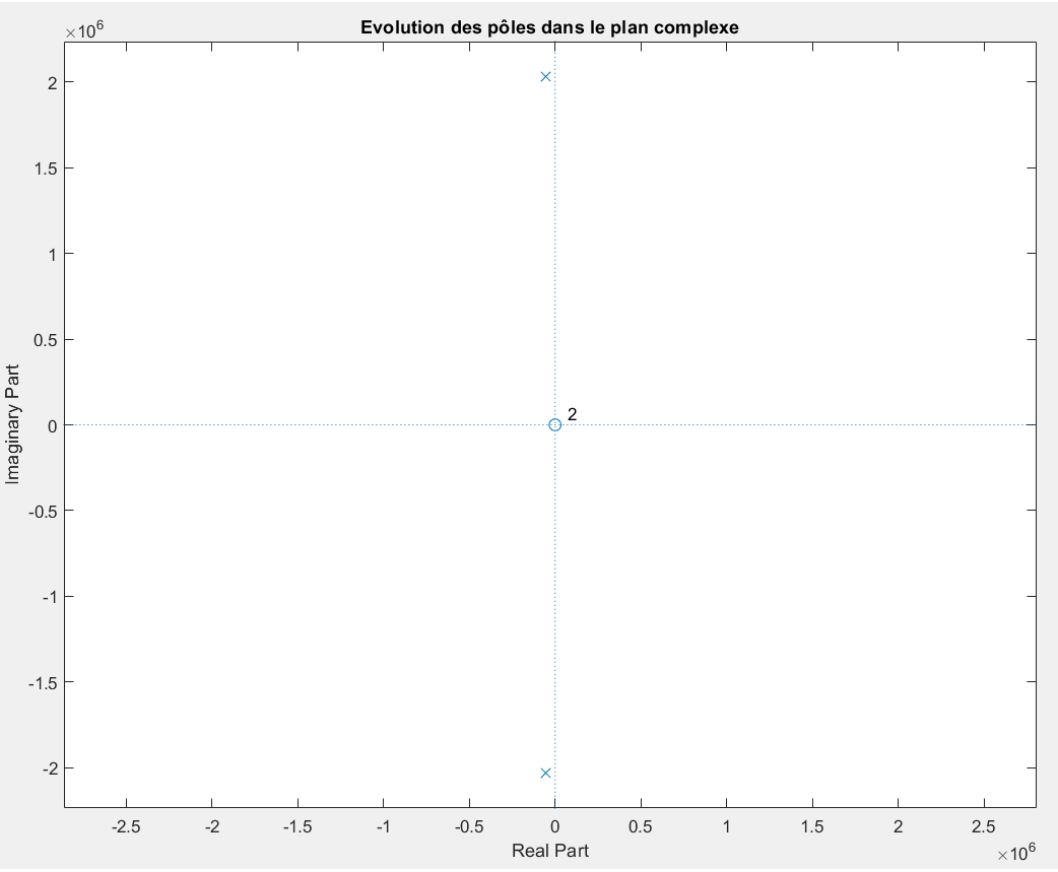


Figure 37 : Évolution des pôles dans le plan complexe

2.6 Diagramme de Bode du transfert de U(p) vers $U_{GT}(p)$

Nous pouvons facilement faire un diagramme de Bode sur Matlab de la fonction de transfert en gardant les hypothèses précédentes via les lignes suivantes :

```
H = tf([Komega*Kc],[J*L, J*R, Ke*Kc]);
bode(H)
```

Figure 38 : Diagramme de Bode du transfert de U(p) vers $U_{GT}(p)$

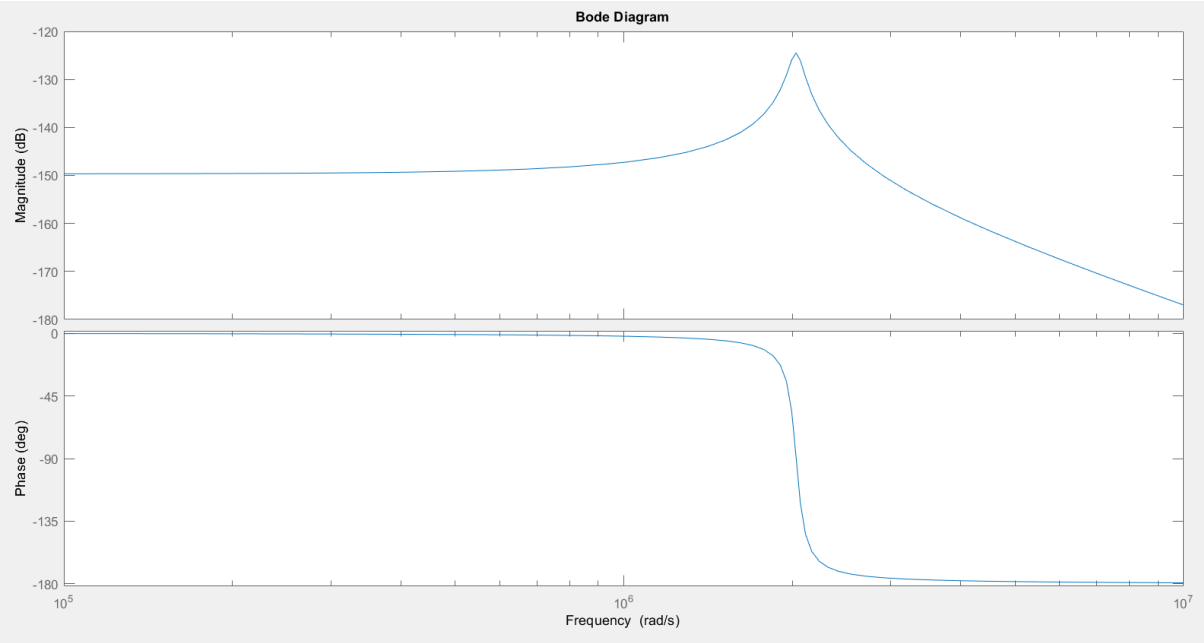


Figure 39 : Diagramme de Bode du transfert de U(p) vers $U_{GT}(p)$

2.7 Diagramme de Bode du transfert de $P_r(p)$ vers $U_{GT}(p)$

On sait que :

$$\frac{U_{GT}(p)}{P_r(p)} = -\frac{\frac{K_\Omega * R}{J * L} + \frac{K_\Omega * p}{J}}{\frac{K_c * K_e}{J * L} + \frac{R}{L} * p + p^2}$$

Nous pouvons facilement faire un diagramme de Bode sur Matlab de la fonction de transfert en gardant les hypothèses précédentes via les lignes suivantes :

```
H = tf([-Komega/J,(-Komega*R)/(J*L)],[1,R/L,(Kc*Ke)/(J*L)]);
bode(H)
```

Figure 40 : Diagramme de Bode du transfert de $P_r(p)$ vers $U_{GT}(p)$

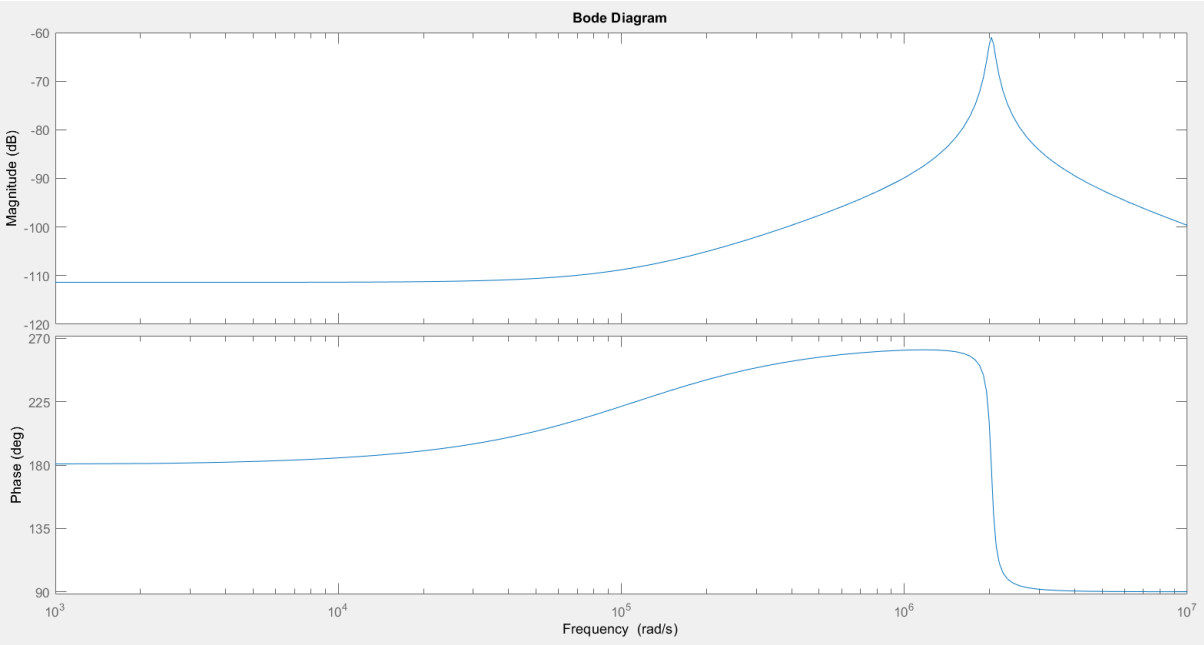


Figure 41 : Diagramme de Bode du transfert de $P_r(p)$ vers $U_{GT}(p)$

2.8 Interprétation et conclusion

Comme nous pouvons le voir sur ces deux diagrammes de Bode du transfert de U(p) vers $U_{GT}(p)$ et du transfert $P_r(p)$ vers $U_{GT}(p)$. Les fréquences de coupure $w_c = 2*\pi*f_c$ sont identiques. Cela est logique parce que le dénominateur est identique. La seule différence entre ces deux fonctions de transfert est que l'une possède un « **p** » au numérateur.

De plus, nous savons qu'un zéro négatif ajoute 90° et 20 dB/décade et qu'un zéro positif ajoute -90° et 20 dB/décade ce qui explique les diagrammes de Bode et de phase précédent.

3. Travail demandé

3.1 Transfert de vitesse

3.1.1 Représentez le schéma-bloc du correcteur RST

Voici les différentes structures pour l'asservissement avec correcteur RST :

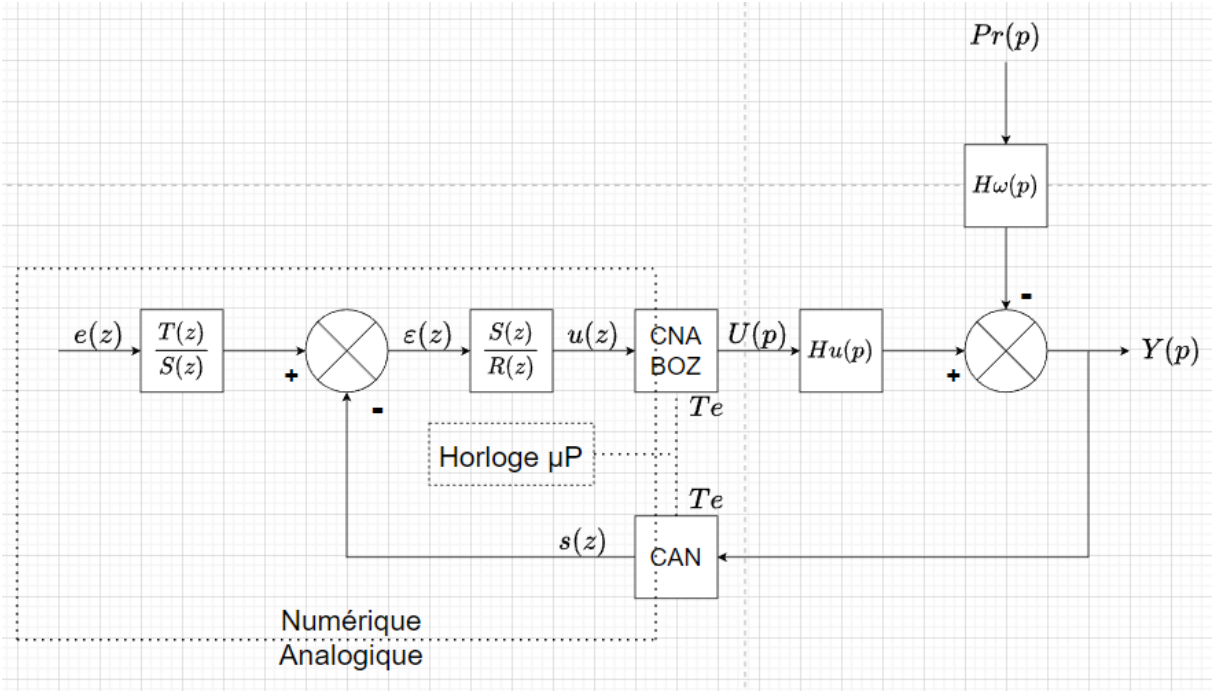


Figure 42 : Structure de commande RST avec correction dans chaîne directe et perturbation en entrée

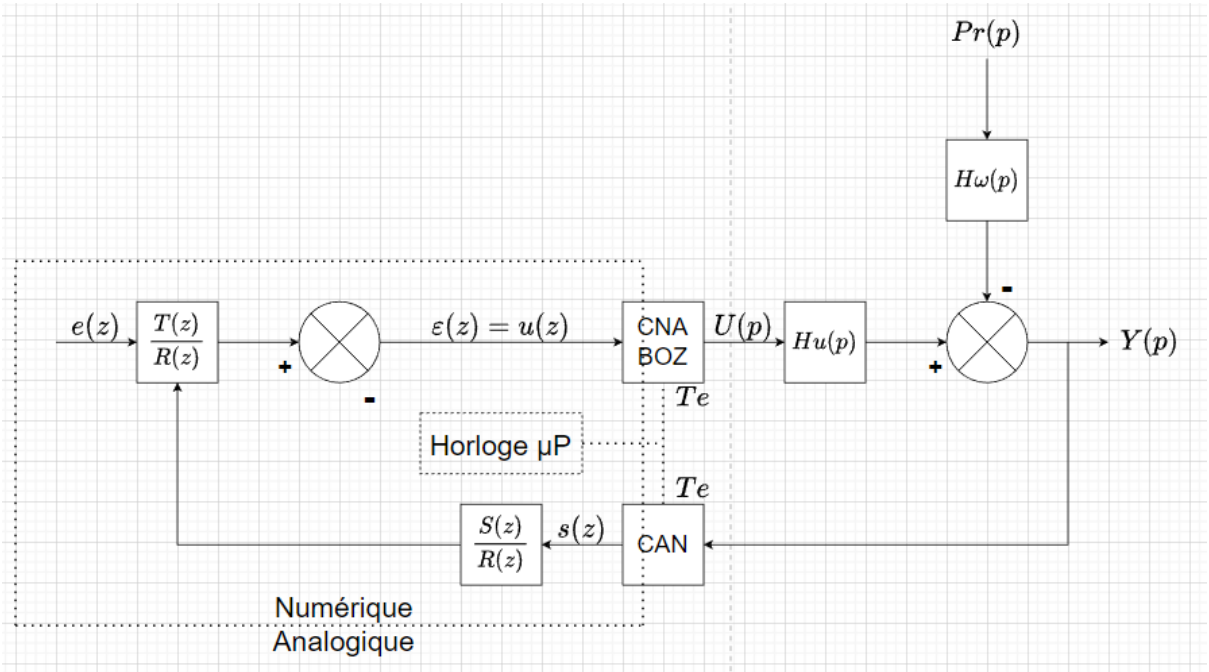


Figure 43 : Structure de commande RST avec correction dans chaîne de retour et perturbation en entrée

3.1.2 Établir un cahier des charges

On a :

$Y(Z) = Z\{Y(k)\}$ où $Y(k) = U_{GT}(t)|t = kTe$

$Y(Z) = G_\Omega(z) * U(z) - \frac{\tilde{K}_v}{K_v} * G_\Omega(z) * W(z)$ où $G_\Omega(Z) = \frac{K_v*(1-D)}{z-D}$, $D = e^{Te*p_{méca}}$ et $W(z) = \frac{W_0*z}{z-1}$

Lorsque l'on a des perturbations d'amplitude constante inconnu, il est alors nécessaire de concevoir un correcteur RST.

D'après l'énonce, le temps de réponse sera choisi 2 à 5 fois plus rapide que le comportement en boucle ouverte donc :

$p_{1_c} = 2$ à $5\ fois\ p_{méca}$

$p_{1_d} = e^{Te*p_{1_c}}$

$\Pi_d(z) = (z - p_{1_d}), q = 1$

$B_d(z) = K$

$A_0(z) = (z - p_{0_d}), p_{0_d} = e^{Te*p_{0_c}}$ avec $|p_{0_c}| > |p_{1_c}|$ et $< \frac{\pi}{Te}$

3.1.3 Calcul de l'ordre minimal du correcteur RST ainsi que les valeurs numériques des polynômes R(z), S(z) et T(z)

$R(z) = (z-1) * \tilde{R}(z) \rightarrow \tilde{R}(z) = 1$

$S(z) = s_1 * z + s_0$ où $s_1 = \frac{1+D-p_{0_d}-p_{1_d}}{K_v*(1-D)}$ et $s_0 = \frac{p_{0_d}*p_{1_d}-D}{K_v*(1-D)}$

$T(z) = \frac{K}{K_v * (1-D)}(z - p_{0_d}) \rightarrow K = (1 - p_{1_d})$

$z = e^{Te*p}$ donc $p = \frac{1}{Te} * ln(z)$ et $\tilde{z} = e^{\widetilde{Te}*p} = e^{\frac{\widetilde{Te}}{Te}*ln(z)} = z^{\frac{\widetilde{Te}}{Te}}$

Partie Pratique

4 Mise en œuvre pratique

4.1 L'identification du système

L'objectif de l'identification est de trouver une estimation de $\hat{\theta}$, du vecteur Θ à partir des échantillons d'entrée $u(k)$ et de sortie $y(k)$ récupérées expérimentalement. Idéalement l'estimation de $\hat{\theta}$ doit être la moins sensible au bruit affectant le modèle. $\hat{\theta}$ doit correspondre à la vraie valeur des paramètres pour une structure de modèle bruité donnée. On parle d'estimateur sans biais (ou non biaisé). Lorsque la structure est de type ARX (Auto-Regressive with eXogenous input), voici le schéma bloc correspondant et l'expression temporelle détaillée :

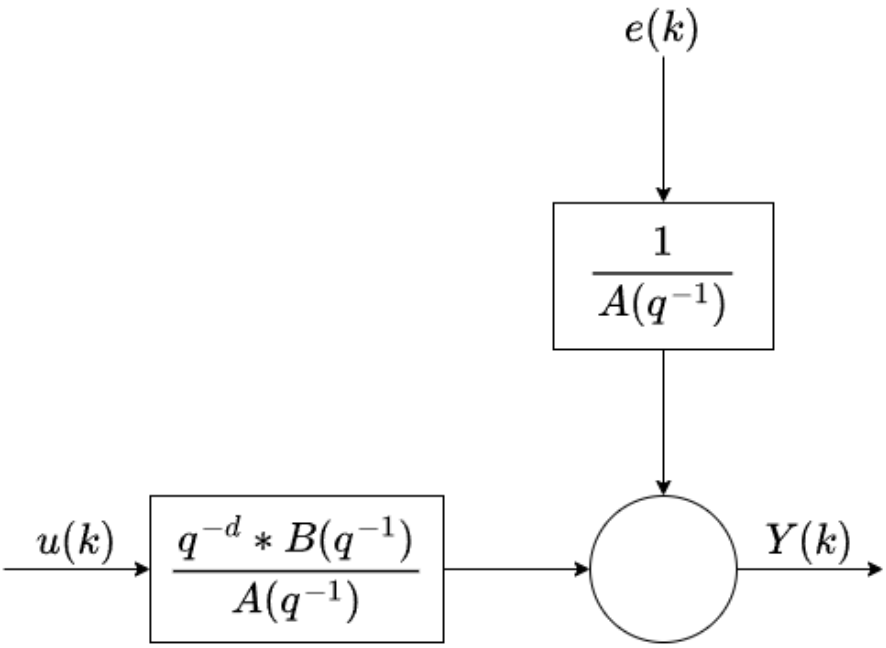


Figure 44 : Schéma bloc correspondant à une structure ARX

Avec :
- $u(k)$ notre signal d'entrée
- $e(k)$ le bruit perturbant notre signal
- $y(k)$ notre signal de sortie
- $A(q^{-1}) = 1 + a_1 * q^{-1} + \ldots + a_{n_A} * q^{-n_A}$ avec $n_A \in \mathbb{N}$
- $B(q^{-1}) = b_0 + b_1 * q^{-1} + \ldots + b_{n_B} * q^{-n_B}$ avec $n_B \in \mathbb{N}*$ et $d \in \mathbb{N}$ où d est le retard pur entre $u(k)$ et $y(k)$

Nous obtenons alors l'expression temporelle détaillée suivante :
$y(k) = -a_1 * y(k-1) - \ldots - a_{n_A} * y(k-n_A) + b_1 u(k-d-1) + \ldots + b_{n_B} u(k-d-n_B) + e(k)$
avec $-a_1 * y(k-1) - \ldots - a_{n_A} * y(k-n_A) + b_1$ qui correspond à la partie auto-régressive, $b_1 u(k-d-1) + \ldots + b_{n_B} u(k-d-n_B)$ à la partie eXogène et $e(k)$ l'influence instantanée du bruit

Nous pouvons maintenant utiliser l'algorithme des moindres carrés récursifs qui est une méthode d'estimation de paramètres qui permet de minimiser l'erreur de prédiction entre les valeurs observées et les valeurs prédites par un modèle. Cette méthode est utilisée pour ajuster des modèles à partir de données en temps réel. Voici l'algorithme pour une structure de type ARX :

$\theta = \begin{bmatrix} a_1 \\ \vdots \\ a_{n_A} \\ b_1 \\ \vdots \\ b_{n_B} \end{bmatrix} \in \mathbb{R}^{n_A+n_B+1}$ et $\varphi(k-1) = \begin{bmatrix} -y(k-1) \\ \vdots \\ -y(k-n_A) \\ u(k-d-1) \\ \vdots \\ u(k-d-n_B) \end{bmatrix}$ le vecteur des observations

Et l'on a montré que :
$\hat{\theta}(N) = argMin\ \Sigma(y(k) - \hat{y}(k|k-1,\Theta))^2 = (\Sigma\varphi(k-1) * \varphi(k-1)^T)^{-1} * \Sigma\varphi(k-1) * y(k)$ où $\hat{\theta}(N)$ dépend du nombre d'itérations

On a vu que :
$Y(p) = \frac{K}{1+z*p} * U(p) - \frac{\tilde{K}}{1+z*p} * W(p)$ et après discrétisation $H(z) = \frac{\alpha*z}{z-\beta} \rightarrow n_A = n_B = 1$ et $d = 0$

On a donc ce code Matlab qui implémente un algorithme récursif des moindres carrés pour identifier une structure ARX à partir de données expérimentales d'un servomécanisme. Le système est représenté par l'équation définie précédemment :

```
% TP Servomécanisme
% M2 SETI

clear all
close all
clc
format short g

cd('C:\Temp\UE520')

Te = 1E-4;
monAmplitude = 10; % en volts
maPeriode = 8; % en secondes
largeurImpulsion = 0.5; % en %
monRetard = 1; % en secondes

% récupération des données expérimentales
load('C:\Temp\UE520\TP_SETI_IdentBO_01.mat')

tTemps = TP_SETI_IdentBO_01.X.Data;
monEntree = TP_SETI_IdentBO_01.Y(1).Data;
maSortie = TP_SETI_IdentBO_01.Y(2).Data;

figure(1);
clf;
plot(tTemps, monEntree, 'b')
hold on
plot(tTemps, maSortie, 'r')
xlabel('Temps(sec)')
ylabel('Signal num')
hold off
```

```
na = 1;
nb = 1;
dd = 0;

somme1 = zeros(na+nb+1,na+nb+1);
somme2 = zeros(na+nb+1,1);

nzpad = max ([na,nb+dd]);
NN = length(maSortie);
for kk = nzpad+1:NN
    phi = [-maSortie(kk-1:-1:kk-na)'; monEntree(kk-dd:-1:kk-dd-nb)'];
    somme1 = somme1 + phi*phi';
    somme2 = somme2 + phi*maSortie(kk);
end

thetaChapeau = somme1\somme2;
coeffA = thetaChapeau(1:na);
coeffB = thetaChapeau(na+1:na+nb+1);

num = [coeffB'];
den = [1,coeffA'];

z=tf('z',Te);
sys_d = tf(num,den,Te);
sys_d.inputDelay=dd;

tTemps_d = 0:Te:(NN-1)*Te;
figure(1);
hold on;
lsim(sys_d,monEntree,tTemps_d);
hold off;
```

Figure 45 : Code de l'algorithme récursif des moindres carrés pour identifier une structure ARX

Ainsi, le code commence par définir les paramètres du système, tels que la période d'échantillonnage, l'amplitude, la largeur de l'impulsion de l'entrée, le retard… Ensuite, il charge les données expérimentales d'entrée et de sortie à partir d'un fichier.

La boucle for calcule les matrices « **somme1 »** et « **somme2 »**, qui sont utilisées pour l'algorithme des moindres carrés récursifs. Les matrices « **somme1** » et « **somme2** » sont calculées à partir des données expérimentales d'entrée et de sortie, ainsi que des coefficients « **na** », « **nb** » et « **dd** ». L'ordre de l'ARX est défini par « **na** » pour l'ordre du dénominateur, « **nb** » pour l'ordre du numérateur.

Une fois que les coefficients du modèle ARX ont été identifiés, le code construit un modèle de système discret en utilisant la fonction de transfert « **tf** » de MATLAB. Le modèle discret est ensuite simulé à l'aide de la fonction « **lsim »** pour obtenir la réponse du système à l'entrée expérimentale.

Finalement, le code trace les données expérimentales d'entrée et de sortie ainsi que la réponse simulée du système pour les comparer graphiquement.

Nous pouvons donc voir les données expérimentales avec l'entrée en bleu et la sortie en rouge pour le servomécanisme :

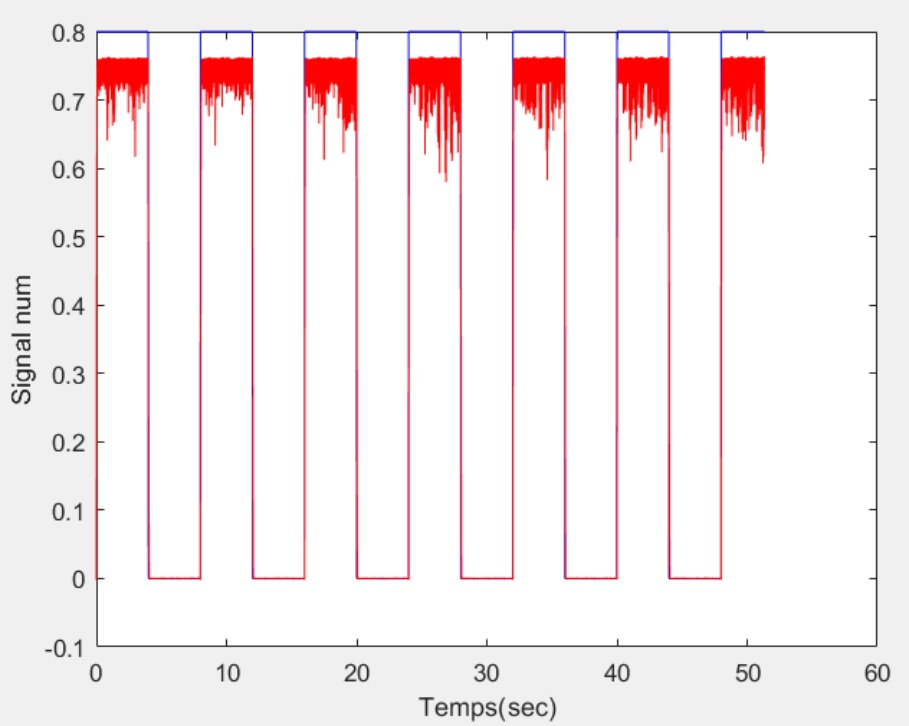


Figure 46 : Entrée et sortie et de notre servomécanisme au cours du temps

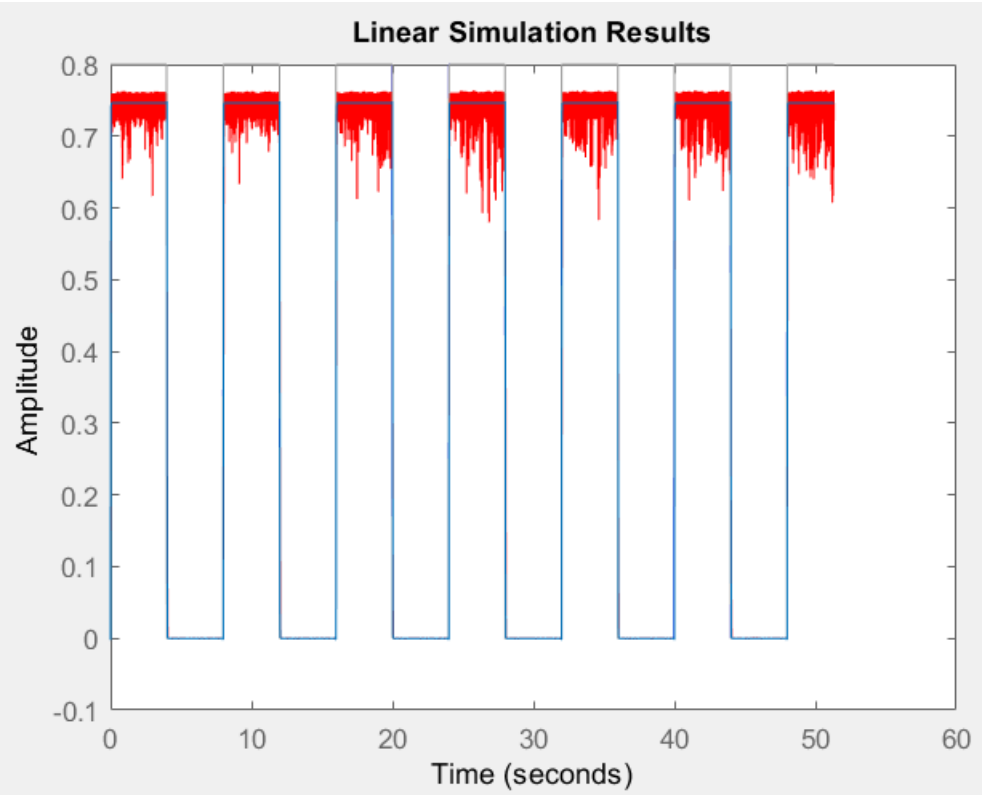


Figure 47 : Entrée, sortie et estimateur (réponse du système à l'entrée expérimentale) de notre servomécanisme au cours du temps

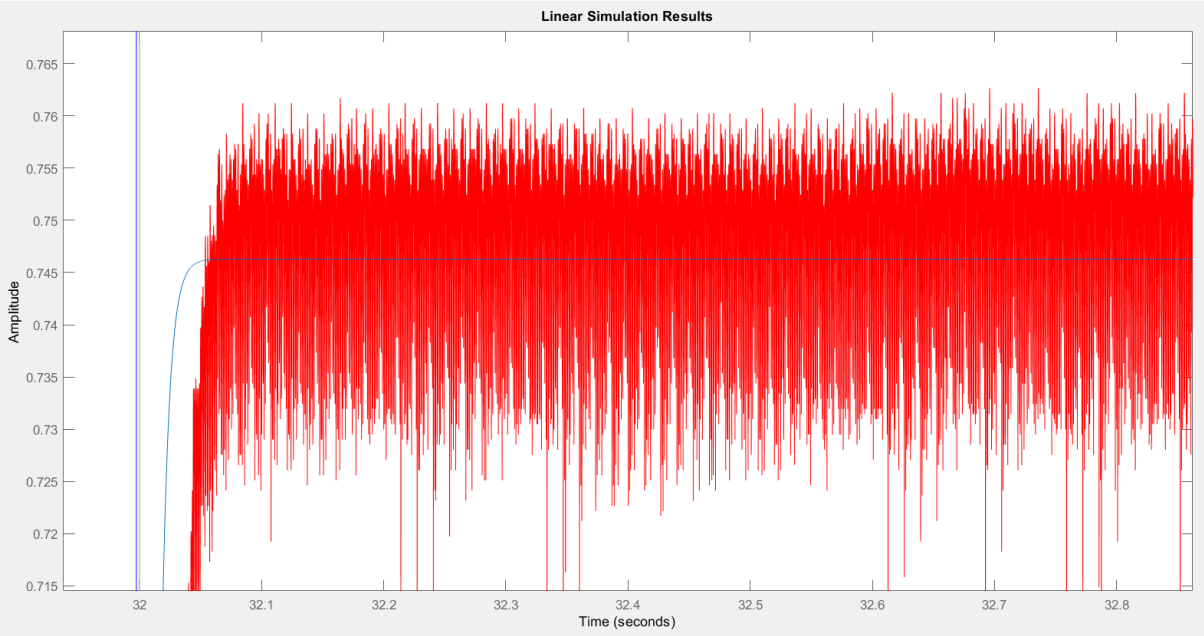


Figure 48 : Entrée, sortie et estimateur (réponse du système à l'entrée expérimentale) de notre servomécanisme au cours du temps

Nous pouvons voir l'estimateur en bleu qui est la réponse du système à l'entrée expérimentale. Cette estimateur semble s'approximer par la moyenne des résultats de la sortie rouge provenant des données expérimentales.

4.2 Modèle de commande du système

L'objectif de la commande d'un système après avoir effectué l'identification est de réguler la sortie du système en fonction de l'entrée. Une fois que le modèle ARX du système a été identifié, il peut être utilisé pour concevoir un régulateur qui génère une commande d'entrée pour le système en fonction de la sortie désirée.

Le régulateur peut être conçu en utilisant différentes méthodes de contrôle, telles que le régulateur PID (proportionnel-intégral-dérivé) ou le régulateur RST. La commande peut être implémentée sur le système en utilisant différentes techniques de contrôle-commande, telles que le contrôle en boucle ouverte, le contrôle en boucle fermée, le contrôle prédictif…

Voici les différentes étapes que nous devons réaliser afin de faire un prédicteur à 1 coup d'avance pour notre système :

On a : $y(k) = G(q^{-1}) * u(k) + H(q^{-1}) * e(k)$

Si l'on applique cela à la structure ARX :

On sait que : $G(q^{-1}) = q^{-d} * \frac{B(q^{-1})}{A(q^{-1})}$ et $H(q^{-1}) = \frac{1}{A(q^{-1})}$ d'où $y(k) = q^{-d} * \frac{B(q^{-1})}{A(q^{-1})} * u(k) + \frac{1}{A(q^{-1})} * e(k)$

$\hat{y}(k|k-1,\hat{\theta}) = q^{-d} * B(q^{-1}) * u(k) + (1 - A(q^{-1}))y(k)$

$\hat{y}(k|k-1,\hat{\theta}) = -a_1 * y(k-1) - \ldots - a_{n_A} * y(k-n_A) + b_1 u(k-d-1) + \ldots + b_{n_B} u(k-d-n_B)$

$\hat{y}(k|k-1,\hat{\theta}) = \theta^T * \varphi(k-1)$

où $\theta^T = [a_1 \ldots a_{n_A}\ b_1 \ldots b_{n_B}]$, $\theta \in \mathbb{R}^{n_A+n_B}$ et $\varphi(k-1) = \begin{bmatrix} -y(k-1) \\ \vdots \\ -y(k-n_A) \\ u(k-d-1) \\ \vdots \\ u(k-d-n_B) \end{bmatrix}$ $\varphi \in \mathbb{R}^{n_A+n_B}$

Après avoir réalisé l'identification du système ainsi que le prédicteur à un coup d'avance. Nous pouvons maintenant réaliser un filtrage des données et dont le but est de prétraiter les données pour éliminer les hautes fréquences qui pourraient biaiser l'identification du système. En effet, les hautes fréquences sont souvent associées à des bruits et des perturbations qui peuvent masquer la véritable réponse du système. Le filtrage permet donc d'améliorer la qualité des données et de faciliter l'identification du modèle.

Pour cela, nous réalisons un filtrage des données via un filtre numérique de type passe-bas (filtre de Butterworth) avec une fréquence de coupure à ajuster. On a donc :

$F(p) = \frac{1}{\frac{p^2}{w_c{}^2} + \frac{2*\xi}{w_c} * p + 1}$ qui est un filtre analogique du 2nd ordre de gain statique unitaire

Nous choisissons $\xi = 0.71$, $w_c = 2 * \pi * f_c$ avec comme exemple $f_c = 500hz$

On peut ensuite numériser ce filtre via les équations de Tustin qui nous dise que $p = \frac{2}{Te} * \frac{z-1}{z+1}$

$$H(z) = F(p)\ avec\ p\ = \frac{2}{Te} * \frac{z-1}{z+1} = \frac{1}{\frac{(2/Te)^2}{{w_c}^2} * (\frac{z-1}{z+1})^2 + \frac{2*\xi}{w_c} * \frac{2}{Te} * \frac{z-1}{z+1} + 1}$$

Donc $H(z) = \frac{(z+1)^2}{\alpha(z-1)^2+\beta(z^2-1)+(z+1)^2}$ où $\alpha = (\frac{2}{Te*w_c})^2$ et $\beta = \frac{4*\xi}{Te*w_c}$

$$H(z) = \frac{z^2+2*z+1}{(\alpha+\beta+1)*z^2+(2-2\alpha)*z+(\alpha-\beta+1)} = \frac{\frac{1}{(\alpha+\beta+1)}*(z^2+2*z+1)}{z^2+\frac{2*(1-\alpha)}{\alpha+\beta+1}*z+\frac{\alpha-\beta+1}{\alpha+\beta+1}}$$

Soit $y_f(k) = H(q^{-1}) * y(k)$ et $u_f(k) = H(q^{-1}) * u(k)$

$$H(q^{-1}) = \frac{\frac{1}{(\alpha+\beta+1)}*(1+2*q^{-1}+q^{-2})}{1+\frac{2*(1-\alpha)}{\alpha+\beta+1}*q^{-1}+\frac{\alpha-\beta+1}{\alpha+\beta+1}*q^{-2}} = \frac{C(q^{-1})}{D(q^{-1})}$$

Supposons que u_f et y_f vérifient l'ARX :

$y_f(k) = \frac{B}{A} * u_f * (k-d) + \frac{1}{A} * \tilde{e}(k)$

$Hy(k) = \frac{B}{A} * Hu(k) + \frac{1}{A} * \tilde{e}(k)$

$y(k) = \frac{B}{A} * u(k) + \frac{1}{A*H} * \tilde{e}(k)$

Ce qui amène au code Matlab suivant qui regroupe l'identification du système, le modèle de commande avec le filtrage et la prédiction des données :

```
clear all
close all
clc
format short g
cd('C:\Temp\UE520\')

% Periode d'echantillonnage
Te = 1E-4;
MonAmplitude = 10; % en Volt
MaPeriode = 8; % en seconde
LargeurImpulsion = 0.5; % fraction de la periode
MonRetard = 1; % en seconde

% Recuperation des donnees experimentales
load('C:\Temp\UE520\TP_SETI_IdentBO_01')

ttemps = TP_SETI_IdentBO_01.X.Data;
MonEntree = TP_SETI_IdentBO_01.Y(1).Data;
MaSortie = TP_SETI_IdentBO_01.Y(2).Data;

figure(1);clf;
plot(ttemps,MonEntree,'b')
hold on
plot(ttemps,MaSortie,'r')
xlabel('temps (sec)')
ylabel('Signal Num')
hold off

% Initialisation de l'estimateur MC, de l'ARX
```

```
Na = 4;
Nb = 4;
dd = 0;
Somme1 = zeros(Na+Nb,Na+Nb);
Somme2 = zeros(Na+Nb,1);

Nzpad = max([Na,Nb+dd]);
NN = length(MaSortie);
for kk = Nzpad+1:NN
    Phi = [-MaSortie(kk-1:-1:kk-Na)';...
        MonEntree(kk-dd-1:-1:kk-dd-Nb)'];
    Somme1 = Somme1 + Phi*Phi';
    Somme2 = Somme2 + Phi*MaSortie(kk);
end

ThetaChapeau = Somme1\Somme2
CoefA = ThetaChapeau(1:Na)
CoefB = ThetaChapeau(Na+1:Na+Nb)

Num = [0,CoefB']
Den = [1,CoefA']

z=tf('z',Te)
Sys_d = tf(Num,Den,Te);Sys_d.InputDelay=dd;

ttemps_d = 0:Te:(NN-1)*Te;
figure(1);
hold on;
lsim(Sys_d,MonEntree,ttemps_d);
hold off

Phi = zeros(Na+Nb,1);
SortiePrediteARX = zeros(1,NN);
ErreurPredictionARX = zeros(1,NN);
for kk = Nzpad+1:NN
    Phi = [-MaSortie(kk-1:-1:kk-Na)';...
        MonEntree(kk-dd-1:-1:kk-dd-Nb)'];
    SortiePrediteARX(kk) = Phi'*ThetaChapeau;
    ErreurPredictionARX(kk) = MaSortie(kk)-...
        SortiePrediteARX(kk);
end
SortieModeleCde = (lsim(Sys_d,MonEntree,ttemps_d))';
MaCol1 = [010,150,220]./255; % Couleur vert fonce
figure(2);clf
hold on
pl0 = plot(ttemps_d,MaSortie,'r-');
pl1 = plot(ttemps_d,SortieModeleCde,'g-',...
    'LineWidth',0.5);
pl2 = plot(ttemps_d,SortiePrediteARX,'-','Color',...
    MaCol1,'LineWidth',1);

% Trace de l'histogramme de l'erreur de prediction
figure(3);clf
hist(ErreurPredictionARX,ceil(NN/1000))
title('erreur de prediction')
% Trace de l'histogramme de l'erreur de simulation
figure(4);clf
hist((MaSortie-SortieModeleCde),ceil(NN/1000))
title('erreur de simulation')

% moyenne des Erreurs (relative)
mean(ErreurPredictionARX/...
    max(abs(ErreurPredictionARX)))
mean(MaSortie-SortieModeleCde)/...
    max(abs(MaSortie-SortieModeleCde))

% Filtrage des donnees E/S par
```

```
% un filtre passe-bas du 2nd ordre
xxi = 0.71;
ffc = 40; % Hz
aalp = (2/(Te*2*pi*ffc))^2;
bbet = 4*xxi/(Te*2*pi*ffc);
hh0 = (aalp-bbet+1)/(aalp+bbet+1);
hh1 = 2*(1-aalp)/(aalp+bbet+1);
FPB1 = inv(aalp+bbet+1)*(z^2+2*z+1)/...
    (z^2+hh1*z+hh0)
figure(5);clf;bode(FPB1,'b');grid on

MonEntree_f = (lsim(FPB1,MonEntree))';
MaSortie_f = (lsim(FPB1,MaSortie))';

figure(6);clf;
plot(ttemps,MonEntree,'b-');
hold on
plot(ttemps,MonEntree_f,'c-'); hold off

figure(7);clf;
plot(ttemps,MaSortie,'r-');
hold on
plot(ttemps,MaSortie_f,'k-'); hold off

Somme1_f = zeros(Na+Nb,Na+Nb);
Somme2_f = zeros(Na+Nb,1);

Nzpad = max([Na,Nb+dd])
NN = length(MaSortie_f);
for kk = Nzpad+1:NN
    Phi = [-MaSortie_f(kk-1:-1:kk-Na)';...
        MonEntree_f(kk-dd-1:-1:kk-dd-Nb)'];
    Somme1_f = Somme1_f + Phi*Phi';
    Somme2_f = Somme2_f + Phi*MaSortie_f(kk);
end

ThetaChapeau_f = Somme1_f\Somme2_f
CoefA_f = ThetaChapeau_f(1:Na)
CoefB_f = ThetaChapeau_f(Na+1:Na+Nb)

Num_f = [0,CoefB_f']
Den_f = [1,CoefA_f']

z=tf('z',Te)
Sys_d_f = tf(Num_f,Den_f,Te);Sys_d_f.InputDelay=dd;

ttemps_d = 0:Te:(NN-1)*Te;
figure(10);
plot(ttemps,MonEntree,'b')
hold on
plot(ttemps,MaSortie,'r')
xlabel('temps (sec)')
ylabel('Signal Num')
lsim(Sys_d_f,MonEntree,ttemps_d);
hold off

Phi = zeros(Na+Nb,1);
SortiePrediteARX_f = zeros(1,NN);
ErreurPredictionARX_f = zeros(1,NN);
for kk = Nzpad+1:NN
    Phi = [-MaSortie(kk-1:-1:kk-Na)';...
        MonEntree(kk-dd-1:-1:kk-dd-Nb)'];
    SortiePrediteARX_f(kk) = Phi'*ThetaChapeau_f;
    ErreurPredictionARX_f(kk) = MaSortie(kk)-...
        SortiePrediteARX_f(kk);
end
SortieModeleCde_f = (lsim(Sys_d_f,MonEntree,ttemps_d))';
```

```
MaCol1 = [010,150,220]./255; % Couleur vert fonce
figure(12);clf
hold on
pl0 = plot(ttemps_d,MaSortie,'r-');
pl1 = plot(ttemps_d,SortieModeleCde_f,'g-',...
    'LineWidth',0.5);
pl2 = plot(ttemps_d,SortiePrediteARX_f,'-','Color',...
    MaCol1,'LineWidth',1);

% Trace de l'histogramme de l'erreur de prediction
figure(13);clf
hist(ErreurPredictionARX_f,ceil(NN/1000))
title('erreur de prediction')
% Trace de l'histogramme de l'erreur de simulation
figure(14);clf
hist((MaSortie-SortieModeleCde_f),ceil(NN/1000))
title('erreur de simulation')

% moyenne des Erreurs (relative)
mean(ErreurPredictionARX_f/...
    max(abs(ErreurPredictionARX_f)))
mean(MaSortie-SortieModeleCde_f)/...
    max(abs(MaSortie-SortieModeleCde_f))
```

Figure 49 : Code Matlab pour l'identification et le modèle de commande du système

Si nous séparons ce code en différentes parties et que nous les analysons, voici ce que nous obtenons :

```
Phi = zeros(Na+Nb,1);
SortiePrediteARX = zeros(1,NN);
ErreurPredictionARX = zeros(1,NN);
for kk = Nzpad+1:NN
    Phi = [-MaSortie(kk-1:-1:kk-Na)';...
        MonEntree(kk-dd-1:-1:kk-dd-Nb)'];
    SortiePrediteARX(kk) = Phi'*ThetaChapeau;
    ErreurPredictionARX(kk) = MaSortie(kk)-...
        SortiePrediteARX(kk);
end
SortieModeleCde = (lsim(Sys_d,MonEntree,ttemps_d))';
MaCol1 = [010,150,220]./255; % Couleur vert fonce
figure(2);clf
hold on
pl0 = plot(ttemps_d,MaSortie,'r-');
pl1 = plot(ttemps_d,SortieModeleCde,'g-',...
    'LineWidth',0.5);
pl2 = plot(ttemps_d,SortiePrediteARX,'-','Color',...
    MaCol1,'LineWidth',1);

% Trace de l'histogramme de l'erreur de prediction
figure(3);clf
hist(ErreurPredictionARX,ceil(NN/1000))
title('erreur de prediction')
% Trace de l'histogramme de l'erreur de simulation
figure(4);clf
hist((MaSortie-SortieModeleCde),ceil(NN/1000))
title('erreur de simulation')

% moyenne des Erreurs (relative)
mean(ErreurPredictionARX/...
    max(abs(ErreurPredictionARX)))
mean(MaSortie-SortieModeleCde)/...
    max(abs(MaSortie-SortieModeleCde))
```

Figure 50 : Code d'un prédicteur à un coup d'avance pour une structure ARX

Cette partie du code réalise la prédiction à un pas en avant de la sortie du système. La matrice « **Phi »** est construite à partir des dernières valeurs de la sortie passées « **MaSortie** » et de l'entrée « **MonEntree** » dans la fenêtre de prédiction, en remontant dans le temps jusqu'à l'instant actuel « **kk** ». La prédiction de la sortie est obtenue en multipliant cette matrice par les coefficients estimés du modèle « **ThetaChapeau** » qui ont été obtenus lors de la phase d'identification du modèle ARX.

La prédiction est ensuite comparée à la sortie réelle du système à l'instant **kk**, et l'erreur de prédiction est stockée (dans le vecteur « **ErreurPredictionARX** »). Cela permet de quantifier la qualité de la prédiction effectuée par le modèle ARX.

Enfin, les figures 2, 3 et 4 permettent de visualiser et de quantifier la qualité de la prédiction et de la simulation. La figure 2 affiche la sortie réelle du système (en rouge) ainsi que la prédiction effectuée par le modèle ARX (en bleu).

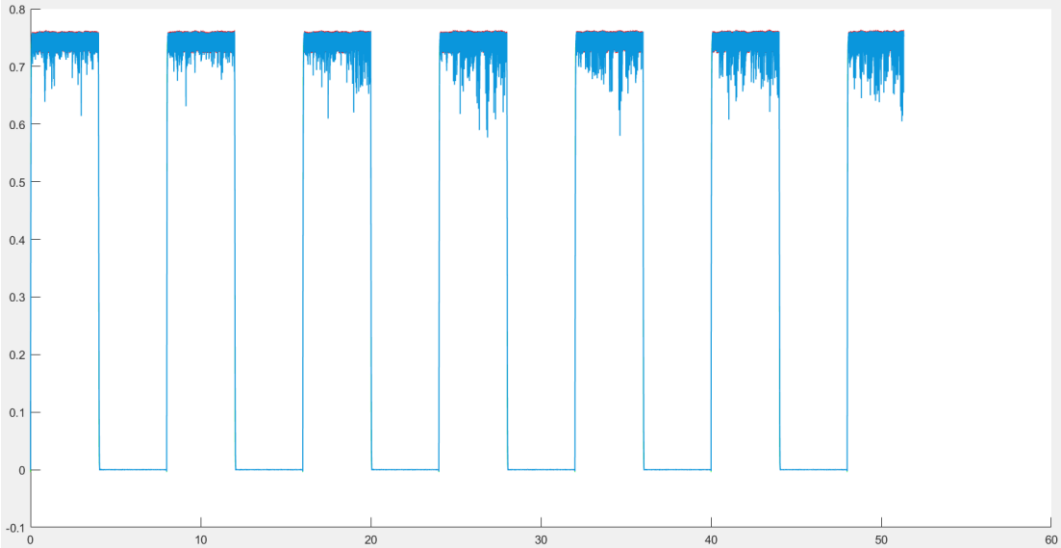


Figure 51 : « Figure 2 » : sortie réelle (en rouge) et la prédiction par le modèle (en bleu)

La figure 3 affiche l'histogramme des erreurs de prédiction (c'est-à-dire la différence entre la sortie réelle et la sortie prédite).

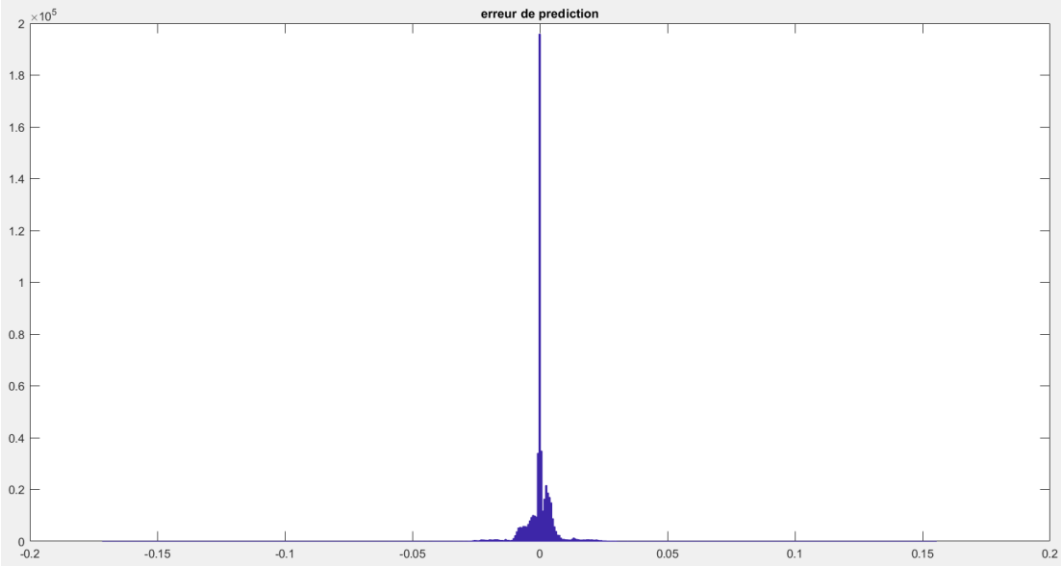


Figure 52 : « Figure 3 » : histogramme des erreurs de prédiction

La figure 4 affiche l'histogramme des erreurs de simulation (c'est-à-dire la différence entre la sortie réelle et la sortie simulée par le modèle ARX).

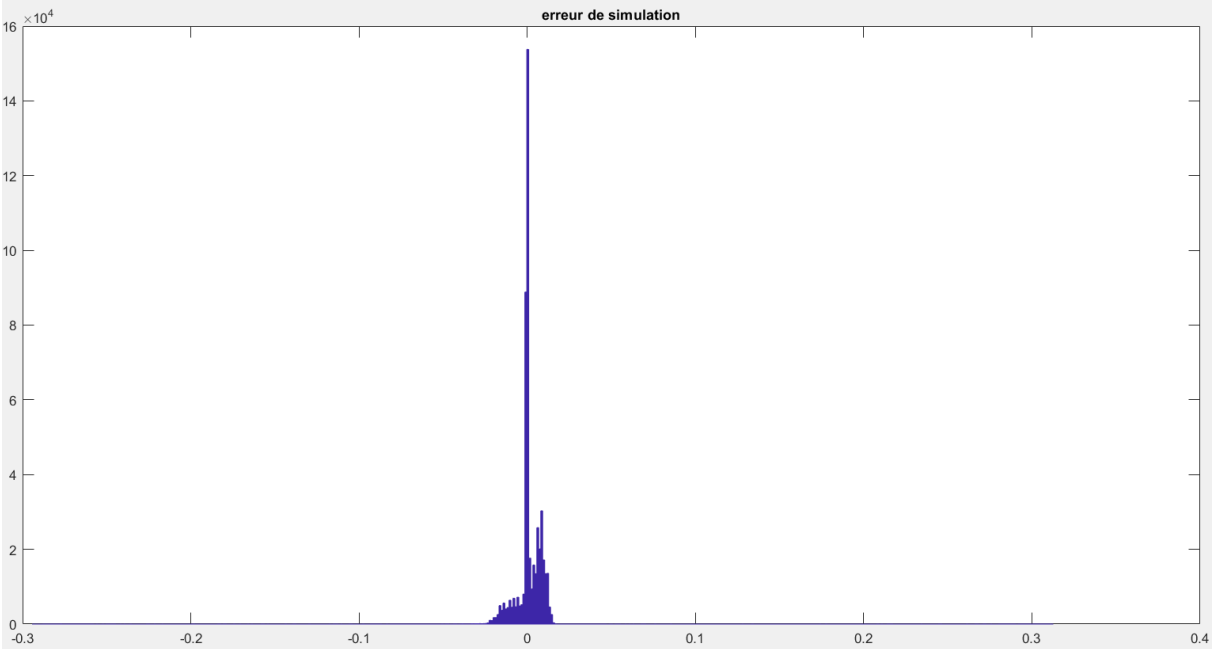


Figure 53 : « Figure 4 » : histogramme des erreurs de simulation

Enfin, les deux dernières lignes du code calculent la moyenne relative des erreurs de prédiction et de simulation.

Voici la deuxième partie importante qui concerne le filtrage des données d'entrée et des sorties obtenues expérimentalement :

```
% Filtrage des donnees E/S par
% un filtre passe-bas du 2nd ordre
xxi = 0.71;
ffc = 40; % Hz
aalp = (2/(Te*2*pi*ffc))^2;
bbet = 4*xxi/(Te*2*pi*ffc);
hh0 = (aalp-bbet+1)/(aalp+bbet+1);
hh1 = 2*(1-aalp)/(aalp+bbet+1);
FPB1 = inv(aalp+bbet+1)*(z^2+2*z+1)/...
    (z^2+hh1*z+hh0)
figure(5);clf;bode(FPB1,'b');grid on

MonEntree_f = (lsim(FPB1,MonEntree))';
MaSortie_f = (lsim(FPB1,MaSortie))';

figure(6);clf;
plot(ttemps,MonEntree,'b-');
hold on
plot(ttemps,MonEntree_f,'c-'); hold off

figure(7);clf;
plot(ttemps,MaSortie,'r-');
hold on
plot(ttemps,MaSortie_f,'k-'); hold off
```

Figure 54 : Application d'un filtre passe-bas du 2ème ordre afin de pré-traiter les données

Cette partie du code applique un filtre passe-bas du 2ème ordre à la fois sur les données d'entrée et de sortie de la fonction à identifier. Le filtre utilisé est défini par ses coefficients « **aalp** », « **bbet** », « **hh0** », « **hh1** », qui sont calculés en fonction de la fréquence de coupure « **ffc** » et du coefficient d'amortissement « **xxi** » spécifiés. Le filtre est ensuite implémenté sous la forme d'une fonction de transfert FPB1 puis est représentée via son diagramme de Bode et son diagramme de Phase.

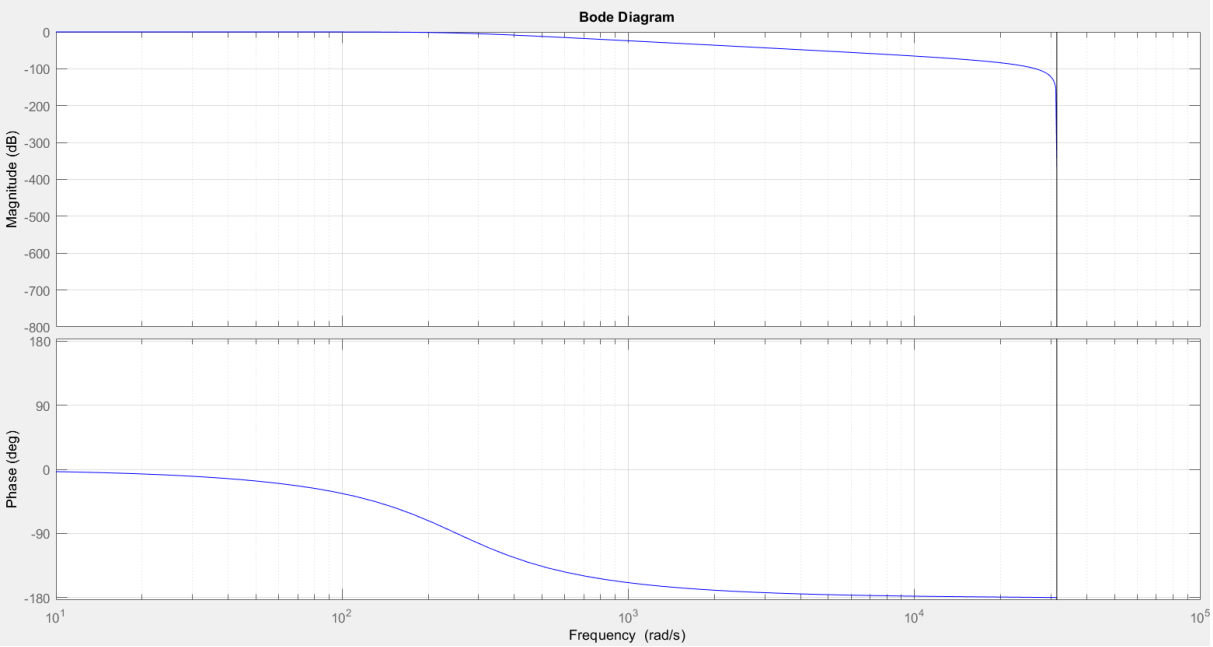


Figure 55 : Diagramme de Bode et de Phase du filtre de pré-traitement des données

Le filtrage des données d'entrée et de sortie est réalisé à l'aide de la fonction « **lsim** » qui permet de simuler la réponse du système en utilisant le filtre défini. Les signaux d'entrée et de sortie filtrés sont stockés dans les variables « **MonEntree_f** » et « **MaSortie_f** ».

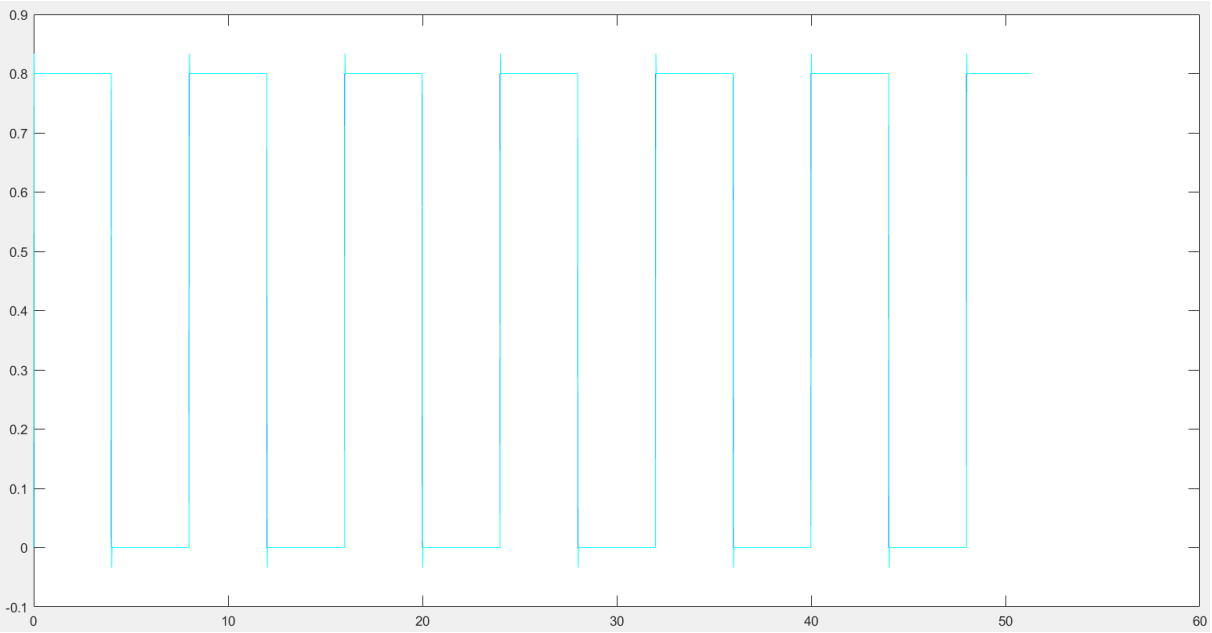


Figure 56 : Entrée réelle (en bleu) et entrée réelle après application du filtre (en cyan)

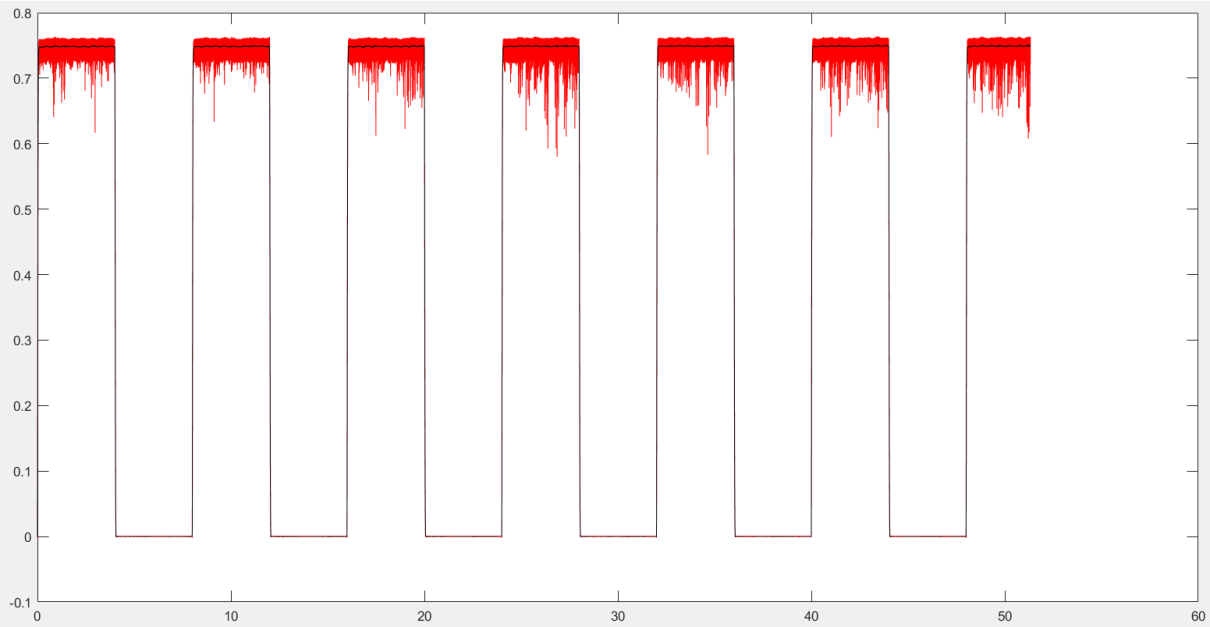


Figure 57 : Sortie réelle (en rouge) et sortie réelle après application du filtre (en noir)

Le but de ce filtrage est de prétraiter les données pour éliminer les hautes fréquences qui pourraient biaiser l'identification du système. En effet, les hautes fréquences sont souvent associées à des bruits et des perturbations qui peuvent masquer la véritable réponse du système. Le filtrage permet donc d'améliorer la qualité des données et de faciliter l'identification du modèle.

Finalement la troisième partie du code qui est logique et assez facile à comprendre concerne l'identification du système. Cependant, pour cette identification, nous avons utilisé les entrées et sorties expérimentales mais après application du filtre passe-bas détaillée précédemment :

```
Somme1_f = zeros(Na+Nb,Na+Nb);
Somme2_f = zeros(Na+Nb,1);

Nzpad = max([Na,Nb+dd])
NN = length(MaSortie_f);
for kk = Nzpad+1:NN
    Phi = [-MaSortie_f(kk-1:-1:kk-Na)';...
        MonEntree_f(kk-dd-1:-1:kk-dd-Nb)'];
    Somme1_f = Somme1_f + Phi*Phi';
    Somme2_f = Somme2_f + Phi*MaSortie_f(kk);
end

ThetaChapeau_f = Somme1_f\Somme2_f
CoefA_f = ThetaChapeau_f(1:Na)
CoefB_f = ThetaChapeau_f(Na+1:Na+Nb)

Num_f = [0,CoefB_f']
Den_f = [1,CoefA_f']

z=tf('z',Te)
Sys_d_f = tf(Num_f,Den_f,Te);Sys_d_f.InputDelay=dd;

ttemps_d = 0:Te:(NN-1)*Te;
figure(10);
```

```
plot(ttemps,MonEntree,'b')
hold on
plot(ttemps,MaSortie,'r')
xlabel('temps (sec)')
ylabel('Signal Num')
lsim(Sys_d_f,MonEntree,ttemps_d);
hold off

Phi = zeros(Na+Nb,1);
SortiePrediteARX_f = zeros(1,NN);
ErreurPredictionARX_f = zeros(1,NN);
for kk = Nzpad+1:NN
    Phi = [-MaSortie(kk-1:-1:kk-Na)';...
        MonEntree(kk-dd-1:-1:kk-dd-Nb)'];
    SortiePrediteARX_f(kk) = Phi'*ThetaChapeau_f;
    ErreurPredictionARX_f(kk) = MaSortie(kk)-...
        SortiePrediteARX_f(kk);
end
SortieModeleCde_f = (lsim(Sys_d_f,MonEntree,ttemps_d))';
MaCol1 = [010,150,220]./255; % Couleur vert fonce
figure(12);clf
hold on
pl0 = plot(ttemps_d,MaSortie,'r-');
pl1 = plot(ttemps_d,SortieModeleCde_f,'g-',...
    'LineWidth',0.5);
pl2 = plot(ttemps_d,SortiePrediteARX_f,'-','Color',...
    MaCol1,'LineWidth',1);

% Trace de l'histogramme de l'erreur de prediction
figure(13);clf
hist(ErreurPredictionARX_f,ceil(NN/1000))
title('erreur de prediction')
% Trace de l'histogramme de l'erreur de simulation
figure(14);clf
hist((MaSortie-SortieModeleCde_f),ceil(NN/1000))
title('erreur de simulation')

% moyenne des Erreurs (relative)
mean(ErreurPredictionARX_f/...
    max(abs(ErreurPredictionARX_f)))
mean(MaSortie-SortieModeleCde_f)/...
    max(abs(MaSortie-SortieModeleCde_f))
```

Figure 58 : Identification du système après filtrage des données d'entrée et de sortie

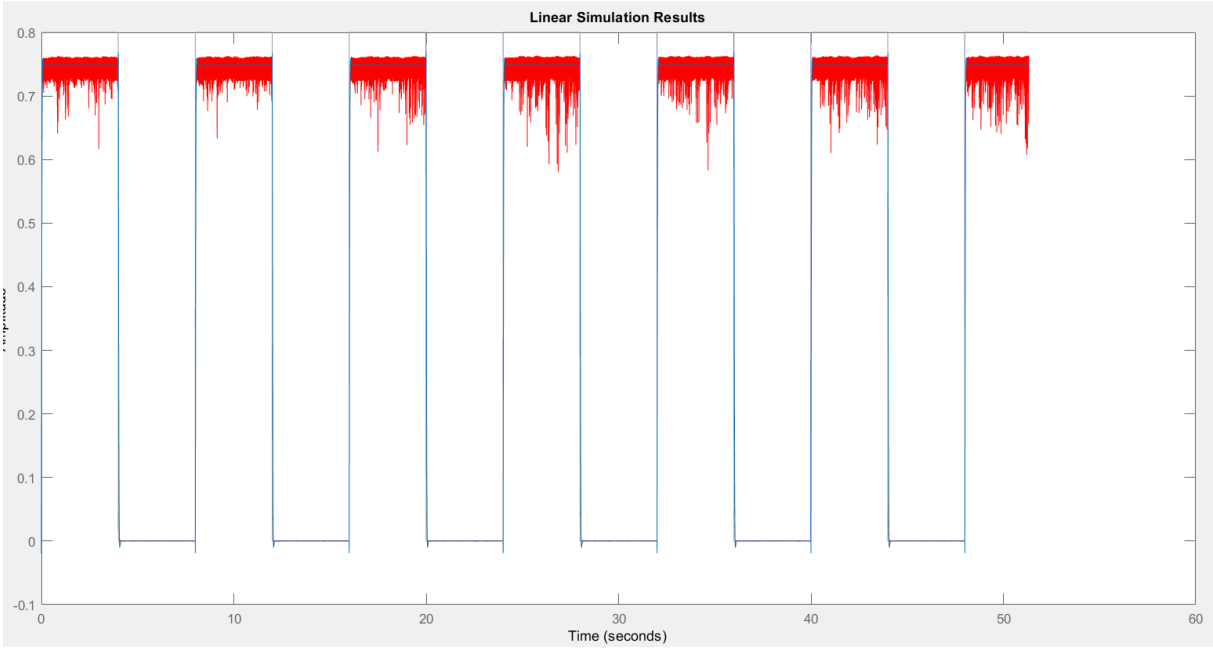


Figure 59 : Entrée et sortie et de notre servomécanisme au cours du temps

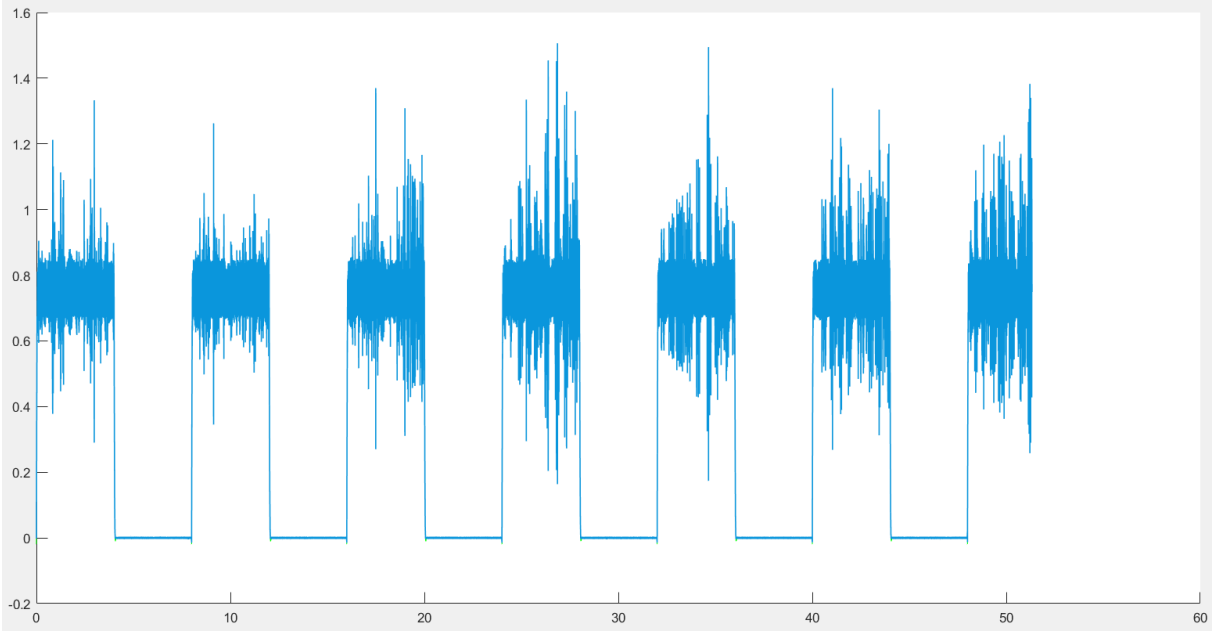


Figure 60 : Comparaison entre les données expérimentales de la sortie (en rouge) et deux sorties prédites (la sortie filtrée du modèle de commande (en vert) et la sortie filtrée prédite par le modèle ARX (en bleu))

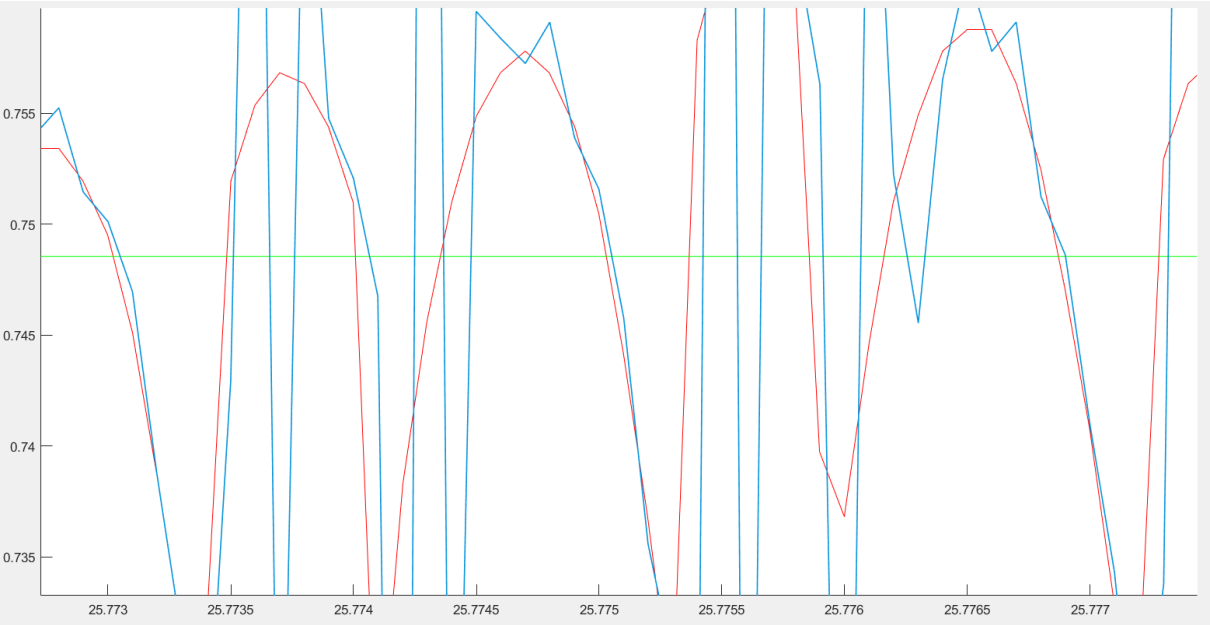


Figure 61 : Comparaison entre les données expérimentales de la sortie (en rouge) et deux sorties prédites (la sortie filtrée du modèle de commande (en vert) et la sortie filtrée prédite par le modèle ARX (en bleu)) après zoom

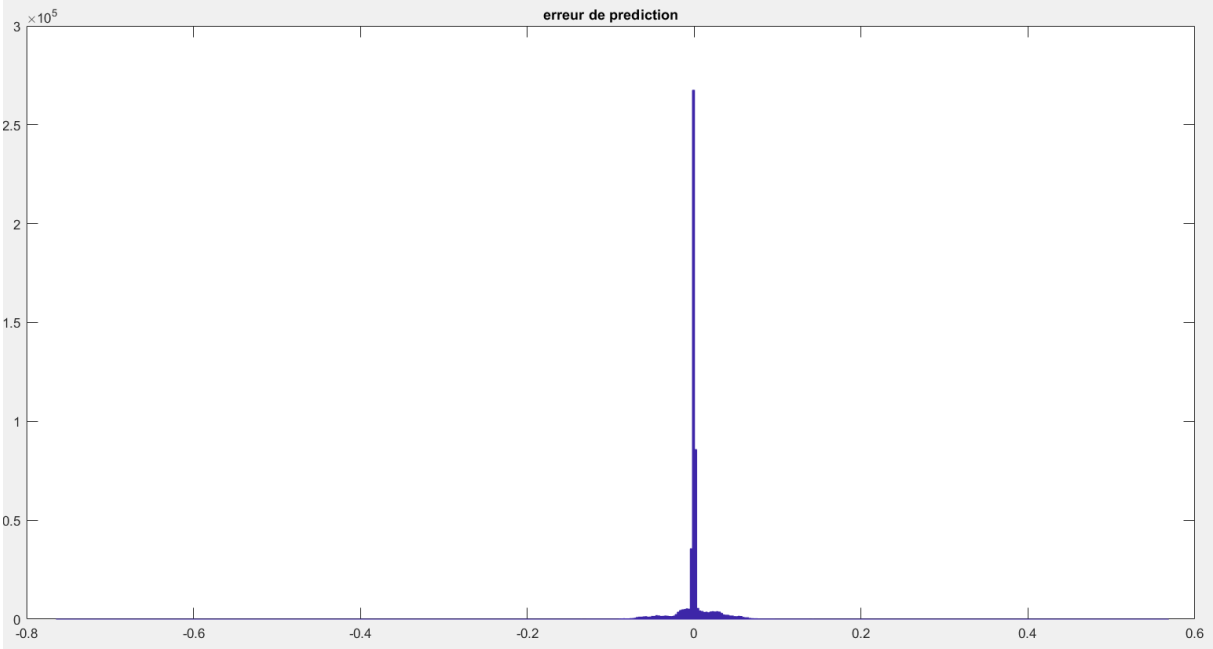


Figure 62 : histogramme des erreurs de prédiction (c'est-à-dire la différence entre la sortie réelle filtrée et la sortie simulée par le modèle ARX)

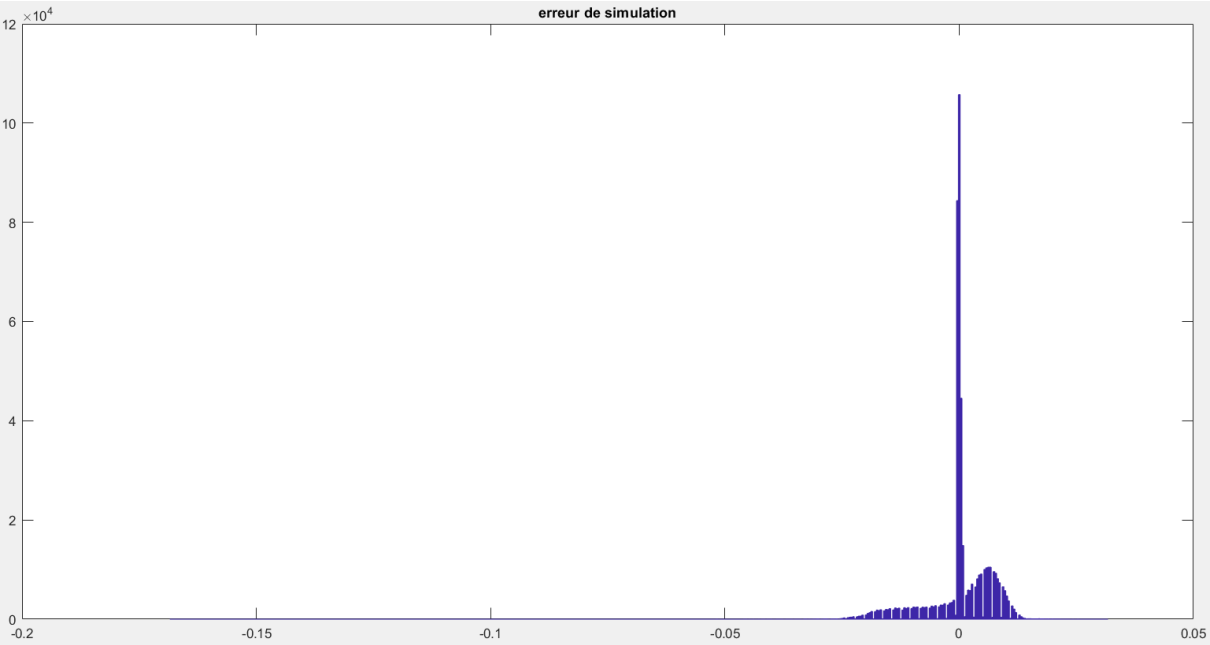


Figure 63 : histogramme des erreurs de simulation (c'est-à-dire la différence entre la sortie réelle filtrée et la sortie simulée par le modèle ARX)